SUST_Ragebait_Tree

Shahjalal University of Science and Technology

1 Templates and Scripts

1.1 sublime

```
{
  "shell_cmd": "g++ -Wall -std=c++20 -O2
     -DLOCAL $file -o $file_base_name
    && bash -c ’ulimit -S -f 10000;
    timeout 5 ./$file_base_name < input
    .in > output.out 2> error.log’",
  "file_regex": "^(..[^:]*):([0-9]+)
    :?([0-9]+)?:? (.*)$",
  "working_dir": "${file_path}",
  "selector": "source.c++, source.c,
    source.cpp",
  "variants":
  [
    {
      "name": "sanitize",
      "shell_cmd": "g++ -Wall -std=c++20
     -fsanitize=address -fsanitize=
    undefined -DLOCAL $file -o
    $file_base_name && bash -c ’ulimit
    -S -f 10000; timeout 5 ./
    $file_base_name < input.in > output
    .out 2> error.log’"
    }
  ]
}
```

1.2 vs code

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "cp",
      "type": "shell",
      "command": "",
      "args": [
        "g++",
        "-Wall",
        "-std=c++20",
        "-O2",
        "-DLOCAL",
        "\"${file}\"",
        "-o",
        "\"${fileDirname}/${
    fileBasenameNoExtension}\"",
        "&&",
        "\"${fileDirname}/${
    fileBasenameNoExtension}\"",
        "<",
        "input.in",
        ">",
        "output.out",
        "2>",
        "error.log"
      ],
      "group": "build",
      "presentation": {
        "reveal": "silent"
      },
      "problemMatcher": {
        "owner": "cpp",
        "fileLocation": ["relative", "${
    workspaceRoot}"],
        "pattern": {
          "regexp": "^(.*):(\\d+):(\\d+)
    :\\s+(warning|error):\\s+(.*)$",
          "file": 1,
          "line": 2,
          "column": 3,
          "severity": 4,
          "message": 5
        }
      }
    }
  ]
}
```

1.3 Interactor

```
socat EXEC:"./interactor" EXEC:"./soln"
```

1.4 debug.h

```
#include <iostream>
using namespace std;
#define see(x) cerr << #x << ": " << x
     << nl
template <typename... Args>
void debug(Args... args) {
  ((cerr << " " << args), ...) << "\n";
}
```

1.5 Template - Kactl

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i, a, b) for(int i = a; i <
     (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
int main() {
        cin.tie(0)->sync_with_stdio(0);
        cin.exceptions(cin.failbit);
}
```

1.6 Template - Nahdi

```
#include <bits/stdc++.h>
using namespace std;
using vi = vector<int>;
using vii = vector<vi>;
using vs = vector<string>;
using ii = pair<int, int>;
```

```
using vpi = vector<ii>;
using ll = long long;
using vll = vector<ll>;
using pll = pair<ll, ll>;
#define f first
#define s second
#define forn(i, n) for (int i = 0; i < n; i++)
#define fore(i, a, b) for (int i = a; i <= b; i++)
#define for1n(i, n) for (int i = 1; i <= n; i++)
#define all(x) x.begin(), x.end()
#define Flag cout << "Reached here.\n"
#define FASTIO              \
  ios::sync_with_stdio(0); \
  cin.tie(0);              \
  cout.tie(0);
#define pb push_back
#define pbb pop_back
#define sz size
#define rsz resize
#define ins insert
#define MAX 1000000000ll
#define MIN -MAX
#define MOD 1000000007LL
#define clr(x, y) memset(x, y, sizeof(x))
int dx[] = {0, 1, -1, 0};
int dy[] = {1, 0, 0, -1};
void init() {
#ifndef ONLINE_JUDGE
  freopen("output.txt", "w", stdout);
  freopen("input.txt", "r", stdin);
#endif
}
void solve() {}
int main() {
  // init();
  // FASTIO;
  int t;
  cin >> t;
  while (t--) solve();
}
```

1.7 Template - Ibnul

```
#include <bits/stdc++.h>
using namespace std;
// Template
// ================================
// pbds
// #include <ext/pb_ds/assoc_container.hpp>
// #include <ext/pb_ds/tree_policy.hpp>
// using namespace __gnu_pbds;
// template<typename T>
// using ordered_set =  tree<T, null_type, less<T>, rb_tree_tag,
// tree_order_statistics_node_update>;
// Debugging
#ifdef LOCAL
#include "debug.h"
#else
#define debug(...)
#define see(x)
#endif
typedef long long ll;
typedef vector<int> VI;
typedef vector<long long> VLL;
typedef vector<bool> VB;
typedef vector<vector<int>> VVI;
typedef pair<int, int> PI;
typedef pair<ll, ll> PLL;
typedef vector<pair<int, int>> VPI;
#define pb push_back
#define ff first
#define ss second
#define mp make_pair
#define all(a) a.begin(), a.end()
#define revall(a) a.rbegin(), a.rend()
#define loop(i, s, e) for (int i = s; i < e; ++i)
#define inp(v)
for (auto& x : v) cin >> x
#define outp(v, st)
for (int i = st, n = v.size(); i < n; ++i) cout << v[i] << " \n"[i == n - 1]
#define nl "\n"
#define yep cout << "YES\n"
#define nope cout << "NO\n"
#define INF (int)1e9
#define INFL (ll)1e18
#define EPS 1e-9
#define MOD 998244353
// #define MOD 1000000007
#define MAXN 2002
// ================================
void solve(int tc) {}
int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(NULL);
  int t = 1;
  // cin >> t;
  for (int i = 1; i <= t; ++i) solve(i);
#ifdef LOCAL
  cerr << "Execution time: " << 1000.f * clock() / CLOCKS_PER_SEC << " ms." << nl;
#endif
  return 0;
}
```

1.8 Mod Template

```
template <unsigned P>
struct modint {
  unsigned v;
  modint() = default;
  template <class T>
  modint(T x) {
    x %= (signed)P, v = x < 0 ? x + P : x;
  }
  modint operator+() const { return *this; }
  modint operator-() const { return modint(0) - *this; }
  modint inv() const { return assert(v), power(*this, P - 2); }
  modint& operator+=(const modint& q) {
    (v += q.v) < P || (v -= P);
    return *this;
  }
  modint& operator-=(const modint& q) {
    (v -= q.v) < P || (v += P);
    return *this;
  }
  modint& operator*=(const modint& q) {
    v = 1ull * v * q.v % P;
    return *this;
  }
  modint& operator/=(const modint& q) {
    v = 1ull * v * q.inv().v % P;
    return *this;
  }
  friend modint operator+(modint p, const modint& q) { return p += q; }
  friend modint operator-(modint p, const modint& q) { return p -= q; }
  friend modint operator*(modint p, const modint& q) { return p *= q; }
  friend modint operator/(modint p, const modint& q) { return p /= q; }
};
typedef modint<998244353> mint;
typedef modint<1000000007> mint2;
```

2 Number Theory

2.1 mini mod

```
inline ll mod(ll n, ll m = MOD) { return (m + n % m) % m; }
ll modpow(ll n, ll k, ll m = MOD) {
  ll ret = 1;
  n = mod(n, m);
  while (k) {
    if (k & 1) ret = (ret * n) % m;
    n = (n * n) % m;
    k >>= 1;
  }
  return ret;
}
inline ll inv(ll n, ll m = MOD) { return modpow(n, m - 2, m); }
inline ll mul(ll x, ll y, ll m = MOD) { return (mod(x, m) * mod(y, m)) % m; }
inline ll dvd(ll x, ll y, ll m = MOD) { return mul(x, inv(y, m), m); }
```

2.2 Binomial Coeffecient

```
ll fact[MAXN];
void calc_fact(ll m = MOD) {
  fact[0] = 1;
  for (int i = 1; i < MAXN; ++i) {
    fact[i] = (fact[i - 1] * i) % m;
  }
}
ll C(int n, int k, ll m = MOD) {
  if (k > n) return 0;
  return dvd(fact[n], fact[k] * fact[n - k] % m, m);
}
```

2.3 Sieve

```
VI primes;
bool isprime[MAXN];
void sieve() {
  isprime[1] = false;
  for (int i = 2; i < MAXN; ++i) isprime[i] = true;
  for (int i = 2; i < MAXN; ++i) {
    if (!isprime[i]) continue;
    primes.pb(i);
    if (1LL * i * i >= MAXN) continue;
    for (int j = i * i; j < MAXN; j += i) isprime[j] = false;
  }
}
```

2.4 CRT

```
int extended_gcd(ll a, ll b, ll& x, ll& y) {
  if (!b) {
    x = 1, y = 0;
    return a;
  }
  ll g = extended_gcd(b, a % b, x, y);
  swap(x, y);
  y -= x * (a / b);
  return g;
}
ll modinv(ll a, ll m) {
  ll x, y;
  extended_gcd(a, m, x, y);
  x %= m;
  return x < 0 ? x + m : x;
}
ll crt(vector<pair<ll, ll>>& mods) {
  ll mod = 1;
  for (auto& p : mods) mod *= p.ss;
  ll ans = 0;
  for (auto [x, p] : mods) {
    ll m = mod / p;
    m = (m * modinv(m, p)) % mod;
    m = (m * x) % mod;
    ans = (ans + m) % mod;
  }
  return ans;
}
```

3 Dynamic Programming

3.1 SOS DP

```
const int B = 20;
int a[1 << B], f[1 << B], g[1 << B];
for (int i = 1; i <= n; i++) {
  cin >> a[i];
  f[a[i]]++;
  g[a[i]]++;
}
// sum over subsets
for (int i = 0; i < B; i++) {
  for (int mask = 0; mask < (1 << B); mask++) {
    if ((mask & (1 << i)) != 0) {
      f[mask] += f[mask ^ (1 << i)];
```

```
    }
  }
}
// sum over supersets
for (int i = 0; i < B; i++) {
  for (int mask = (1 << B) - 1; mask >=
     0; mask--) {
    if ((mask & (1 << i)) == 0) g[mask]
     += g[mask ^ (1 << i)];
  }
}
```

3.2 CHT

```
struct CHT {
  vector<ll> m, b;
  int ptr = 0;
  bool bad(int l1, int l2, int l3) {
    return 1.0 * (b[l3] - b[l1]) * (m[l1
     ] - m[l2]) <=
          1.0 * (b[l2] - b[l1]) *
              (m[l1] - m[l3]);  //(
     slope dec+query min),(slope inc+
     query max)
    return 1.0 * (b[l3] - b[l1]) * (m[l1
     ] - m[l2]) >
          1.0 * (b[l2] - b[l1]) *
              (m[l1] - m[l3]);  //(
     slope dec+query max), (slope inc+
     query min)}
    void add(ll _m, ll _b) {
      m.push_back(_m);
      b.push_back(_b);
      int s = m.size();
      while (s >= 3 && bad(s - 3, s - 2,
      s - 1)) {
        s--;
        m.erase(m.end() - 2);
        b.erase(b.end() - 2);
      }
    }
    ll f(int i, ll x) { return m[i] * x
     + b[i]; }
    //(slope dec+query min), (slope inc+
     query max) -> x increasing
    //(slope dec+query max), (slope inc+
     query min) -> x decreasing
    ll query(ll x) {
      if (ptr >= m.size()) ptr = m.size
     () - 1;
      while (ptr < m.size() - 1 && f(ptr
      + 1, x) < f(ptr, x)) ptr++;
      return f(ptr, x);
    }
    ll bs(int l, int r, ll x) {
      int mid = (l + r) / 2;
      if (mid + 1 < m.size() && f(mid +
     1, x) < f(mid, x))
        return bs(mid + 1, r, x);  // >
     for max
      if (mid - 1 >= 0 && f(mid - 1, x)
     < f(mid, x))
        return bs(l, mid - 1, x);  // >
     for max
      return f(mid, x);
    }
  };
```

4 Data Structure

4.1 Fenwick Tree

Description: Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value. **Time:** Both operations are $O(\log N)$

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[
     pos] += dif
    for (; pos < s.size(); pos |= pos +
     1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values
     in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res
     += s[pos-1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st
      sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or
      -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1)
      {
      if (pos + pw <= s.size() && s[pos
     + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
};
```

4.2 Fenwick Tree 2D

Description: Computes sums $a[i, j]$ for all $i < I, j < J$, and increases single elements $a[i, j]$. Requires that the elements to be updated are known in advance (call `fakeUpdate()` before `init()`). **Time:** $O(\log^2 N)$. (Use persistent segment trees for $O(\log N)$)

```
struct FT2 {
  vector<vi> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x
     ].push_back(y);
  }
  void init() {
    for (vi& v : ys) sort(all(v)), ft.
     emplace_back(sz(v));
  }
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]),
      y) - ys[x].begin()); }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  }
};
```

4.3 Segment Tree (1 based)

```
struct ST {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  vector<long long> t, a;
  ST(int n) {
    t.resize(4*n);
    a.resize(n);
  }
  ST(vector<long long>& a) : a(n) {
    t.resize(4*n);
  }
  inline long long combine(long long a,
     long long b) {
    return a + b; // Change this for Max
     , Min, XOR, etc.
  }
  inline void pull(int nd) {
    t[nd] = combine(t[lc], t[rc]);
  }
  void build(int nd, int b, int e) {
    if (b == e) {
      t[nd] = a[b];
      return;
    }
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
    pull(nd);
  }
  void upd(int nd, int b, int e, int i,
     long long v) {
    if (b == e) {
      t[nd] = v; // For "add v", use t[
     nd] += v;
      return;
    }
    int mid = (b + e) >> 1;
    if (i <= mid) upd(lc, b, mid, i, v);
    else upd(rc, mid + 1, e, i, v);
    pull(nd);
  }
  long long query(int nd, int b, int e,
     int i, int j) {
    if (i > e || b > j) return 0; //
     Return identity (0 for sum, 1e18
     for min, etc.)
    if (i <= b && e <= j) return t[nd];
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, i,
     j), query(rc, mid + 1, e, i, j));
  }
};
```

4.4 Lazy Propagation (1 based)

```
struct ST {
  #define lc (n << 1)
  #define rc ((n << 1) | 1)
//   long long t[4 * N], lazy[4 * N];
  vector<ll> a, t, lazy;
  ST(vll& a) : a(a) {
    lazy.resize(4*a.size()+1), t.resize
     (4*a.size()+1);
  }
  ST(int n) {
    a.resize(n), lazy.resize(4*n), t.
     resize(4*n);
  }
  inline void push(int n, int b, int e)
     {
    if (lazy[n] == 0) return;
    t[n] = (ll) t[n] + (ll) lazy[n] * (
     ll) (e - b + 1);
    t[n] %= mod;
    if (b != e) {
      lazy[lc] = (ll) lazy[lc] + lazy[n
     ];
      lazy[rc] = (ll) lazy[rc] + lazy[n
     ];
      lazy[rc] %= mod, lazy[lc] %= mod;
    }
    lazy[n] = 0;
  }
  inline long long combine(long long a,
     long long b) {
    return (a + b) % mod;
  }
  inline void pull(int n) {
    t[n] = (t[lc] + t[rc]) % mod;
  }
  void build(int n, int b, int e) {
    lazy[n] = 0;
    if (b == e) {
      t[n] = a[b];
      return;
    }
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
    pull(n);
  }
  void upd(int n, int b, int e, int i,
     int j, long long v) {
    push(n, b, e);
    if (j < b || e < i) return;
    if (i <= b && e <= j) {
      lazy[n] = v; //set lazy
      push(n, b, e);
      return;
    }
    int mid = (b + e) >> 1;
    upd(lc, b, mid, i, j, v);
    upd(rc, mid + 1, e, i, j, v);
    pull(n);
  }
  long long query(int n, int b, int e,
     int i, int j) {
    push(n, b, e);
    if (i > e || b > j) return 0; //
     return null
```

```
    if (i <= b && e <= j) return t[n];
    int mid = (b + e) >> 1;
    return (combine(query(lc, b, mid, i,
      j), query(rc, mid + 1, e, i, j)))
     % mod;
  }
};
```

4.5 Persistent Segment Tree

```
template <typename T>
struct node {
  T val;
  int l, r;
};
template <typename T>
struct persistent_segtree {
  int n;
  T init_val;
  vector<node<T>> nodes;
  vector<int> version;
  function<T(T, T)> f;
  persistent_segtree() {}
  persistent_segtree(int sz, function<T(
     T, T)> f, T val = 0) {
    n = sz;
    this->f = f;
    init_val = val;
    build(vector<T>(n, init_val));
  }
  persistent_segtree(vector<T>& a,
     function<T(T, T)> f, T val = 0) {
    n = a.size();
    this->f = f;
    init_val = val;
    build(a);
  }
  int build(vector<T> a) {
    int root = _build(0, n - 1, a);
    version.pb(root);
    return version.size() - 1;
  }
  int update(int i, T x, int v = -1) {
    if (v == -1) v = version.size() - 1;
    int root = _update(version[v], 0, n
     - 1, i, x);
    version.pb(root);
    return version.size() - 1;
  }
  T get(int l, int r, int v) { return
     _get(version[v], 0, n - 1, l, r); }
  // version is 1 indexed (0 is the
     initial before updates)
  T get_kth(int ver_l, int ver_r, int k)
      {
    return _get_kth(version[ver_l - 1],
     version[ver_r], 0, n - 1, k);
  }
  int _build(int l, int r, vector<T>& a)
      {
    int root = nodes.size();
    nodes.pb({});
    if (l == r) {
      nodes[root].val = a[l];
      return root;
    }
    int mid = (l + r) / 2;
    int new_child = _build(l, mid, a);
    nodes[root].l = new_child;
    new_child = _build(mid + 1, r, a);
    nodes[root].r = new_child;
    nodes[root].val = f(nodes[nodes[root
     ].l].val, nodes[nodes[root].r].val)
     ;
    return root;
  }
  int _update(int u, int l, int r, int i
     , T x) {
    int root = nodes.size();
    node<T> old = nodes[u];
    nodes.pb(old);
    if (l == r) {
      nodes[root].val = x;
      return root;
    }
    int mid = (l + r) / 2;
    if (i <= mid) {
      int newl = _update(nodes[u].l, l,
     mid, i, x);
      nodes[root].l = newl;
    } else {
      int newr = _update(nodes[u].r, mid
     + 1, r, i, x);
      nodes[root].r = newr;
    }
    nodes[root].val = f(nodes[nodes[root
     ].l].val, nodes[nodes[root].r].val)
     ;
    return root;
  }
  T _get(int u, int tl, int tr, int l,
     int r) {
    if (l > tr || r < tl) return
     init_val;
    if (l <= tl && r >= tr) return nodes
     [u].val;
    int mid = (tl + tr) / 2;
    return f(_get(nodes[u].l, tl, mid, l
     , r),
             _get(nodes[u].r, mid + 1,
     tr, l, r));
  }
  int _get_kth(int u, int v, int l, int
     r, int k) {
    if (l == r) return l;
    int mid = (l + r) / 2;
    int kl = nodes[nodes[v].l].val -
     nodes[nodes[u].l].val;
    if (kl >= k)
      return _get_kth(nodes[u].l, nodes[
     v].l, l, mid, k);
    else
      return _get_kth(nodes[u].r, nodes[
     v].r, mid + 1, r, k - kl);
  }
};
```

4.6 Merge Sort Tree

```
// 0-based index
template <typename T>
struct MST {
  int n;
  vector<vector<T>> tree;
  MST(vector<T>& a) : n(a.size()) {
    tree.resize(4 * n);
    build(0, 0, n - 1, a);
  }
  void build(int u, int l, int r, vector
     <T>& a) {
    if (l == r) {
      tree[u].pb(a[l]);
      return;
    }
    int mid = (l + r) / 2;
    build(2 * u + 1, l, mid, a);
    build(2 * u + 2, mid + 1, r, a);
    tree[u].resize(r - l + 1);
    merge(all(tree[2 * u + 1]), all(tree
     [2 * u + 2]), tree[u].begin());
  }
  // Returns number of elements smaller
     than x in range l, r
  int query(int l, int r, T x) { return
     get(0, 0, n - 1, l, r, x); }
  int get(int u, int tl, int tr, int l,
     int r, T x) {
    if (tl >= l && tr <= r)
      return lower_bound(all(tree[u]), x
     ) - tree[u].begin();
    if (tl > r || tr < l) return 0;
    int mid = (tl + tr) / 2;
    return get(2 * u + 1, tl, mid, l, r,
     x) +
           get(2 * u + 2, mid + 1, tr, l
     , r, x);
  }
};
```

4.7 Segment Tree 2D (1 based)

```
// USAGE
ST2D sg(NX, NY); // dimensions 1..NX,
    1..NY
sg.upd   (1, 1, NX, x, y, v);
sg.query (1, 1, NX, x1, x2, y1, y2);
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  #define lcy (ndy << 1)
  #define rcy ((ndy << 1) | 1)
  int NX, NY;
  vector<vector<long long>> t;
  ST2D(int NX, int NY) : NX(NX), NY(NY),
     t(4 * NX, vector<long long>(4 * NY
     , 0)) {}
  inline long long combine(long long a,
     long long b) { return max(a, b); }
  inline void pull_y(int nd, int ndy) {
    t[nd][ndy] = combine(t[nd][lcy], t[
     nd][rcy]);
  }
  inline void pull_x(int nd, int ndy) {
    t[nd][ndy] = combine(t[lc][ndy], t[
     rc][ndy]);
  }
  // ------------------------
  // Inner seg tree on y-axis
  // ------------------------
  void upd_y(int nd, int ndy, int by,
     int ey, int j, long long v, bool
     leaf_x) {
    if (by == ey) {
      if (leaf_x) t[nd][ndy] = v;
      else        t[nd][ndy] = combine(t
     [lc][ndy], t[rc][ndy]);
      return;
    }
    int mid = (by + ey) >> 1;
    if (j <= mid) upd_y(nd, lcy, by, mid
     , j, v, leaf_x);
    else          upd_y(nd, rcy, mid+1,
     ey, j, v, leaf_x);
    pull_y(nd, ndy);
  }
  long long query_y(int nd, int ndy, int
      by, int ey, int y1, int y2) {
    if (y1 > ey || by > y2) return 0;
    if (y1 <= by && ey <= y2) return t[
     nd][ndy];
    int mid = (by + ey) >> 1;
    return combine(query_y(nd, lcy, by,
     mid, y1, y2),
                   query_y(nd, rcy, mid
     +1, ey, y1, y2));
  }
  // ------------------------
  // Outer seg tree on x-axis
  // ------------------------
  void upd(int nd, int bx, int ex, int i
     , int j, long long v) {
    if (bx == ex) {
      upd_y(nd, 1, 1, NY, j, v, true);
      return;
    }
    int mid = (bx + ex) >> 1;
    if (i <= mid) upd(lc, bx, mid, i, j,
      v);
    else          upd(rc, mid+1, ex, i,
     j, v);
    upd_y(nd, 1, 1, NY, j, v, false);
     // pull x into inner nodes
  }
  long long query(int nd, int bx, int ex
     , int x1, int x2, int y1, int y2) {
    if (x1 > ex || bx > x2) return 0;
    if (x1 <= bx && ex <= x2) return
     query_y(nd, 1, 1, NY, y1, y2);
    int mid = (bx + ex) >> 1;
    return combine(query(lc, bx, mid, x1
     , x2, y1, y2),
                   query(rc, mid+1, ex,
     x1, x2, y1, y2));
  }
};
```

4.8 Segment Tree 2D (Index compression)

```
// USAGE
ST2D seg(n);
// Phase 1: register all (x, y) points
    that will ever be updated
for (auto& [x, y, v] : updates)
    seg.collect(1, 1, n, x, y);
```

```
// Phase 2: compress and allocate inner trees
seg.build(1, 1, n);
// Updates and queries
seg.upd   (1, 1, n, x, y, v);
seg.query (1, 1, n, x1, x2, y1, y2);
////////////////////////////////////////

void compress(vector<int>& a) {
    sort(all(a));
    a.erase(unique(all(a)), a.end());
}
int ind(vi& a, int val) {
    return lower_bound(all(a), val) - a.begin();
}
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  int n;
  vector<vi> ys;       // compressed y-coords per x-node
  vector<vi> t;        // inner seg tree per x-node
  ST2D(int n) : n(n), ys(4 * n), t(4 * n) {}
  void collect(int nd, int b, int e, int x, int y) {
    ys[nd].pb(y);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) collect(lc, b, mid, x, y);
    else          collect(rc, mid + 1, e, x, y);
  }
  void build(int nd, int b, int e) {
    compress(ys[nd]);
    t[nd].assign(ys[nd].sz() * 2, 0);
    if (b == e) return;
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
  }
  inline long long combine(long long a, long long b) { return max(a, b); }
  void upd_y(int nd, int y, long long v) {
    int m = ys[nd].sz();
    int i = ind(ys[nd], y) + m;
    t[nd][i] = combine(t[nd][i], v);
    for (i >>= 1; i >= 1; i >>= 1)
      t[nd][i] = combine(t[nd][i*2], t[nd][i*2+1]);
  }
  long long query_y(int nd, int y1, int y2) {
    if (y1 > y2) return 0;
    int m = ys[nd].sz();
    int l = ind(ys[nd], y1) + m;
    int r = (int)(upper_bound(all(ys[nd]), y2) - ys[nd].begin() - 1) + m;
    ll mx = 0;
    while (l <= r) {
        if (l%2 == 1) mx = combine(mx, t[nd][l++]);
        if (r%2 == 0) mx = combine(mx, t[nd][r--]);
        l >>= 1; r >>= 1;
    }
    return mx;
  }
  void upd(int nd, int b, int e, int x, int y, long long v) {
    upd_y(nd, y, v);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) upd(lc, b, mid, x, y, v);
    else          upd(rc, mid + 1, e, x, y, v);
  }
  long long query(int nd, int b, int e, int x1, int x2, int y1, int y2) {
    if (x1 > x2 || y1 > y2) return 0;
    // empty range (e.g. a[i].f == 1)
    if (x1 > e  || b > x2)  return 0;
    if (x1 <= b && e <= x2)  return query_y(nd, y1, y2);
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, x1, x2, y1, y2),
                   query(rc, mid+1, e, x1, x2, y1, y2));
  }
};
```

4.9 DSU

```
struct dsu {
  vector<int> parent, size;
  dsu(int n) {
    parent.resize(n + 1);
    size.resize(n + 1);
    for (int i = 1; i <= n; ++i) {
      parent[i] = i;
      size[i] = 1;
    }
  }
  int find(int u) {
    if (parent[u] == u) return u;
    return parent[u] = find(parent[u]);
  }
  void merge(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      if (size[u] < size[v]) swap(u, v);
      parent[v] = u;
      size[u] += size[v];
    }
  }
};
```

4.10 SQRT Decomposition

```
struct SqrtDecomp {
int n, sz;
    vector<ll> a;
    vector<map<int,int>> b;
    SqrtDecomp(int n, vi& arr) : n(n) {
        sz = sqrt(n) + 1;
        /*...*/
    }
    void upd(int i, ll v) {
        /*...*/
    }
    ll combine(ll s, ll elem, int v) { /*...*/ }
    ll combineBlock(ll s, map<int,int>& blk, int v) {
        /*...*/
    }
    ll query(int l, int r, int v) {
        ll s = 0;
        r--;
        if(l > r) {
            return 0;
        }
        int bl = l/sz, br = r/sz;
        if (bl == br) {
            for (int i = l; i <= r; i++) s = combine(s, a[i], v);
        } else {
            for (int i = l; i < (bl+1)*sz; i++) s = combine(s, a[i], v);
            for (int i = bl+1; i < br; i++) s = combineBlock(s, b[i], v);
            for (int i = br*sz; i <= r; i++) s = combine(s, a[i], v);
        }
        return s;
    }
};
```

4.11 MOs Algorithm

```
void add(int ind, int end) { ... }  // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... }  // remove a[ind]
int calc(){...}                     // compute current answer
vector<int> mo(vector<pair<int, int>> Q) {
  int L = 0, R = 0, blk = 350;  // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first / blk, x.second ^ -(x.first / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t) {
    return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}
```

4.12 MOs with updates

```
/// @note: Store query indices as [l, r). DON'T FORGET TO MAKE R EXCLUSIVE
struct mos_update {
    ll a[100005];
    int L, R; // needed by apply
    ll ans;
    mos_update() { /* ... */ }
    void add(int ind, int end) { /* ... */ }
    void del(int ind, int end) { /* ... */ }
    void apply(int pos, int val) {
        if (L <= pos && pos < R) { del(pos, -1); a[pos] = val; add(pos, -1); }
        else                     { a[pos] = val; }
    }
    // Q[i] = {l, r, t}
    // U[i] = {pos, old_val, new_val}
    vector<ll> mo(vector<vector<int>>& Q, vector<vector<int>>& U, int n) {
        L = 0, R = 0; int T = 0;
        ans = 0;
        int blk = max(1, (int)pow(Q.size(), 0.666));
        vll s(Q.size()), res(Q.size());
        iota(all(s), 0);
        sort(all(s), [&](int i, int j) {
            int li = Q[i][0]/blk, lj = Q[j][0]/blk;
            if (li != lj) return li < lj;
            int ri = Q[i][1]/blk, rj = Q[j][1]/blk;
            if (ri != rj) return (li & 1) ? ri < rj : ri > rj;
            return (ri & 1) ? Q[i][2] < Q[j][2] : Q[i][2] > Q[j][2];
        });
        for (int qi : s) {
            int ql = Q[qi][0], qr = Q[qi][1], qt = Q[qi][2];
            while (T < qt) { auto& u = U[T++]; apply(u[0], u[2]); }
            while (T > qt) { auto& u = U[--T]; apply(u[0], u[1]); }
            while (L > ql) add(--L, 0);
            while (R < qr) add(R++, 1);
            while (L < ql) del(L++, 0);
            while (R > qr) del(--R, 1);
            res[qi] = ans;
        }
        return res;
    }
};
```

4.13 Sparse Table (RMQ)

```
template<typename T>
struct SparseTable {
  const int N = 1e5 + 9;
  vector<vector<T>> t;
  vector<T> a;
  T combine(T x, T y) { return min(x, y); }
  void build(int n) {
    t.assign(n + 1, vector<T>(18));
    for(int i = 1; i <= n; ++i) t[i][0] = a[i];
    for(int k = 1; k < 18; ++k)
```

```
    for(int i = 1; i + (1 << k) - 1 <= n; ++i)
      t[i][k] = combine(t[i][k-1], t[i + (1 << (k-1))][k-1]);
  }
  T query(int l, int r) {
    int k = 31 - __builtin_clz(r - l + 1);
    return combine(t[l][k], t[r - (1 << k) + 1][k]);
  }
};
```

4.14 Prefix Sum 2D

```
template<typename T>
struct PrefixSum2D {
  vector<vector<T>> p;
  int rows, cols;
  T combine(T x, T y) { return x + y; }
  void build(vector<vector<T>>& a) {
    rows = a.size(); cols = a[0].size();
    p.assign(rows + 1, vector<T>(cols + 1, 0));
    for(int i = 1; i <= rows; ++i)
      for(int j = 1; j <= cols; ++j)
        p[i][j] = a[i-1][j-1] + p[i-1][j] + p[i][j-1] - p[i-1][j-1];
  }
  // query sum over [r1,c1]..[r2,c2] (1-indexed)
  T query(int r1, int c1, int r2, int c2) {
    return p[r2][c2] - p[r1-1][c2] - p[r2][c1-1] + p[r1-1][c1-1];
  }
};
```

4.15 Prefix Sum 3D

```
template<typename T>
struct PrefixSum3D {
  vector<vector<vector<T>>> p;
  int X, Y, Z;
  T combine(T x, T y) { return x + y; }
  void build(vector<vector<vector<T>>>& a) {
    X = a.size(); Y = a[0].size(); Z = a[0][0].size();
    p.assign(X+1, vector<vector<T>>(Y+1, vector<T>(Z+1, 0)));
    for(int i = 1; i <= X; ++i)
      for(int j = 1; j <= Y; ++j)
        for(int k = 1; k <= Z; ++k)
          p[i][j][k] = a[i-1][j-1][k-1]
            + p[i-1][j][k] + p[i][j-1][k] + p[i][j][k-1]
            - p[i-1][j-1][k] - p[i-1][j][k-1] - p[i][j-1][k-1]
            + p[i-1][j-1][k-1];
  }
  // query sum over [x1,y1,z1]..[x2,y2,z2] (1-indexed)
  T query(int x1, int y1, int z1, int x2, int y2, int z2) {
    return p[x2][y2][z2]
      - p[x1-1][y2][z2] - p[x2][y1-1][z2] - p[x2][y2][z1-1]
      + p[x1-1][y1-1][z2] + p[x1-1][y2][z1-1] + p[x2][y1-1][z1-1]
      - p[x1-1][y1-1][z1-1];
  }
};
```

5 Graph

5.1 Dijkstra

```
struct Node {
  int u;
  ll dist;
  bool operator<(const Node& v) const {
    return dist > v.dist; }
};
vector<ll> dijkstra(int start, vector<VPI>& adj) {
  vector<ll> dis(adj.size(), INFL);
  priority_queue<Node> q;
  dis[start] = 0;
  q.push({start, 0});
  while (!q.empty()) {
    Node node = q.top();
    int u = node.u;
    q.pop();
    if (dis[u] < node.dist) continue;
    for (auto e : adj[u]) {
      if (node.dist + e.ss < dis[e.ff]) {
        dis[e.ff] = node.dist + e.ss;
        q.push({e.ff, dis[e.ff]});
      }
    }
  }
  return dis;
}
```

5.2 Bellmanford

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get `dist = inf`; nodes reachable through negative-weight cycles get `dist = -inf`. Assumes $V^2 \max |w| < \infty$ to prevent overflow.
Time: $O(VE)$

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w; ll s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };
void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });
  int lim = nodes.size() / 2 + 2; // /3+100 with shuffled vertices
  for(int i = 0; i < lim; i++) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  for(int i = 0; i < lim; i++) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

5.3 Biconnected Component

Description: Finds biconnected components, bridges, and articulation points in a simple undirected graph.
Usage: Build adj list via `ed[u].push_back({v, edge_id})`. Call `bicomps(n, f)` where `f(comp)` is called for each component (vector of edge IDs). After call, `bridges` contains bridge edges and `isArt[i]` is 1 for articulation points.
Time: $O(V+E)$

```
/// USAGE
adj.resize(n);
for(int i = 0; i < edges.size(); i++) {
  auto [u, v] = edges[i];
  adj[u].push_back({v, i});
  adj [v].push_back({u, i});
}
vector<vector<int>> components;
bicomps(n, [&](vi& comp) {
  components.push_back(comp);
});
/////////////////////////////////////
vector<vector<pair<int,int>>> adj;   // adj[u] = {(v, edge_id), ...}
vector<int> num, isArt;
vector<pair<int,int>> bridges;       // {u, v} for each bridge
vector<int> st;            // edge stack
int Time = 0;
template <class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time;
  int top = me;
  int children = 0;
  for (auto [y, e] : adj[at]) {
    if (e == par) continue;
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me) st.push_back(e); // back edge
    } else {
      children++;
      int si = st.size();
      int up = dfs(y, e, f);
      top = min(top, up);
      if (par != -1 && up >= me) isArt[at] = 1;
      if (up > me) bridges.push_back({at, y});
      if (up >= me) {
        st.push_back(e);
        vi comp(st.begin() + si, st.end());
        f(comp);
        st.resize(si);
      } else {
        st.push_back(e);
      }
    }
  }
  if (par == -1 && children > 1) isArt[at] = 1;
  return top;
}
template <class F>
void bicomps(int n, F f) {
  adj.resize(n);
  num.assign(n, 0);
  isArt.assign(n, 0);
  bridges.clear();
  st.clear();
  Time = 0;
  for(int i = 0; i < n; i++) if (!num[i]) dfs(i, -1, f);
}
```

5.4 LCA

```
struct LCA {
  int step = 0, k = 0;
  vector<int> vis_start, vis_end;
  vector<vector<int>> st;
  vector<vector<int>>& tree;
  LCA(vector<vector<int>>& tree) : tree(tree) { init(tree.size() - 1); }
  void build_st(int u, int p) {
    st[u][0] = p;
    for (int i = 1; i <= k; ++i) st[u][i] = st[st[u][i - 1]][i - 1];
  }
  void dfs(int u, int p) {
    vis_start[u] = step++;
    build_st(u, p);
    for (auto v : tree[u]) {
      if (v != p) dfs(v, u);
    }
    vis_end[u] = step++;
  }
  bool is_ancestor(int a, int b) {
    return vis_start[a] < vis_start[b] && vis_end[a] > vis_end[b];
  }
  int get(int a, int b) {
    if (a == b) return a;
    if (is_ancestor(a, b)) return a;
    if (is_ancestor(b, a)) return b;
    for (int i = k; i >= 0; --i) {
      if (!is_ancestor(st[a][i], b)) a = st[a][i];
    }
    return st[a][0];
  }
  void init(int n) {
    vis_start.assign(n + 1, 0);
    vis_end.assign(n + 1, 0);
    k = __lg(2 * n - 1);
    st.assign(n + 1, vector<int>(k + 1));
    dfs(1, 1);
```

```
  }
  // Extra
  int get_ancestor(int u, int n) {
    for (int i = 0; n > 0; ++i) {
      if (n & (1 << i)) {
        u = st[u][i];
        n -= (1 << i);
      }
    }
    return u;
  }
};
```

5.5 SCC

Description: Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage:
`scc(graph, (vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components. `O(E+V)`

```
vector<int> val, comp, z;
int Time, ncomps;
vector<vector<int>> sccs;
int dfs(int j, vector<vector<int>>& g) {
    int low = val[j] = ++Time, x; z.
     push_back(j);
    for (int e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ?: dfs(e,
     g));
    if (low == val[j]) {
        sccs.push_back({});
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            sccs.back().push_back(x);
        } while (x != j);
        ncomps++;
    }
    return val[j] = low;
}
void get_sccs(vector<vector<int>>& g) {
    int n = g.size();
    val.assign(n, 0); comp.assign(n, -1)
     ;
    z.clear(); sccs.clear();
    Time = ncomps = 0;
    for (int i = 0; i < n; i++) if (comp
     [i] < 0) dfs(i, g);
}
```

5.6 HLD

```
vector<int> heavy, chain, label, par, sz
    , dep;
int timer;
void dfsz(int u, int p, vector<vector<
    int>>& tree) {
  sz[u] = 1;
  par[u] = p;
  dep[u] = dep[p] + 1;
  int mx = -1;
  for (int v : tree[u]) {
    if (v == p) continue;
    dfsz(v, u, tree);
    sz[u] += sz[v];
    if (sz[v] > mx) mx = sz[v], heavy[u]
      = v;
  }
}
void dfshld(int u, int p, vector<vector<
    int>>& tree, int head) {
  label[u] = timer++;
  chain[u] = head;
  if (heavy[u] != -1) dfshld(heavy[u], u
    , tree, head);
  for (int v : tree[u]) {
    if (v == p || v == heavy[u])
     continue;
    dfshld(v, u, tree, v);
  }
}
void inithld(vector<vector<int>>& tree)
    {
  int n = tree.size();
  heavy = vector<int>(n, -1);
  chain = vector<int>(n);
  label = vector<int>(n);
  sz = vector<int>(n);
  par = vector<int>(n);
  dep = vector<int>(n);
  timer = 1;
  dfsz(1, 0, tree);
  dfshld(1, 0, tree, 1);
}
ll query(int u, int v /*, fenwick/
    segtree/anything& ds*/) {
  ll ret = 0;  // init value
  for (; chain[u] != chain[v]; v = par[
    chain[v]]) {
    if (dep[chain[u]] > dep[chain[v]])
     swap(u, v);
    // ret ?= ds.query(label[chain[v]],
     label[v]);
  }
  if (dep[u] > dep[v]) swap(u, v);
  // lca is u
  // ret ?= ds.query(label[u], label[v])
    ;
  return ret;
}
// Updating u : ds.update(label[u]...)
```

5.7 Centroid Decomposition

```
struct CD {
  vector<vector<int>>& tree;
  vector<int> par, sz;
  vector<bool> dead;
  CD(vector<vector<int>>& tree) : tree(
    tree) {
    par.resize(tree.size());
    sz.resize(tree.size());
    dead.resize(tree.size());
    build(1, 0);
  }
  void build(int u, int p) {
    dfs(u, p);
    u = dfs(u, p, sz[u]);
    dead[u] = true;
    par[u] = p;
    for (int v : tree[u]) {
      if (!dead[v]) build(v, u);
    }
  }
  void dfs(int u, int p) {
    sz[u] = 1;
    for (int v : tree[u]) {
      if (v == p || dead[v]) continue;
      dfs(v, u);
      sz[u] += sz[v];
    }
  }
  int dfs(int u, int p, int n) {
    for (int v : tree[u]) {
      if (v == p || dead[v]) continue;
      if (sz[v] > n / 2) return dfs(v, u
    , n);
    }
    return u;
  }
  int operator[](int i) { return par[i];
      }
};
```

6 Flow and Matching

6.1 Min Cost Max Flow

Description: Min-cost max-flow. If costs can be negative, call `setpi` before `maxflow`, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $O(FE \log V)$ where F is max flow; $O(VE)$ for `setpi`.

```
const ll INF = numeric_limits<ll>::max()
    / 4;
struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge*> par;
  MCMF(int N) : N(N), ed(N), seen(N),
    dist(N), pi(N), par(N) {}
  void addEdge(int from, int to, ll cap,
     ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{from, to, sz
     (ed[to]), cap, cost, 0});
    ed[to].push_back(edge{to, from, sz(
     ed[from]) - 1, 0, -cost, 0});
  }
  void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0;
    ll di;
    __gnu_pbds::priority_queue<pair<ll,
     int>> q;
    vector<decltype(q)::point_iterator>
     its(N);
    q.push({0, s});
    while (!q.empty()) {
      s = q.top().second;
      q.pop();
      seen[s] = 1;
      di = dist[s] + pi[s];
      for (edge& e : ed[s])
        if (!seen[e.to]) {
          ll val = di - pi[e.to] + e.
    cost;
          if (e.cap - e.flow > 0 && val
    < dist[e.to]) {
            dist[e.to] = val;
            par[e.to] = &e;
            if (its[e.to] == q.end())
              its[e.to] = q.push({-dist[
    e.to], e.to});
            else
              q.modify(its[e.to], {-dist
     [e.to], e.to});
          }
        }
    }
    rep(i, 0, N) pi[i] = min(pi[i] +
     dist[i], INF);
  }
  pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[
     x->from])
        fl = min(fl, x->cap - x->flow);
      totflow += fl;
      for (edge* x = par[t]; x; x = par[
     x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    rep(i, 0, N) for (edge& e : ed[i])
     totcost += e.cost * e.flow;
    return {totflow, totcost / 2};
  }
  // If some costs can be negative, call
      this before maxflow:
  void setpi(int s) {  // (otherwise,
     leave this out)
    fill(all(pi), INF);
    pi[s] = 0;
    int it = N, ch = 1;
    ll v;
    while (ch-- && it--)
      rep(i, 0, N) if (pi[i] !=
                     INF) for (edge& e
       :
                              ed[i])
       if (e.cap) if ((v = pi[i] + e.cost)
       <

                  pi[e.to]) pi[e.to] =
       v,
```

```
                                ch = 1;
    assert(it >= 0);  // negative cost cycle
  }
};
```

6.2 Dinic

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.
Time: $O(VE \log U)$

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

6.3 HopcroftKarp

Description: Fast bipartite matching. `g` is the adjacency list of the left partition. `r` is a vector of -1s of size equal to the right partition; after the call, `r[i]` is the left-side match of right vertex i, or -1 if unmatched. Returns the size of the maximum matching.
Time: $O(E\sqrt{V})$

```
int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i, 0, n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i, 0, t) for (int v : g[q[i]]) {
      if (r[v] == -1)
        f = 1;
      else if (!d[r[v]])
        d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    rep(i, 0, n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
// --- Shortened usage ---
int main() {
    int n, m, k; cin >> n >> m >> k;
    vi r(n + m, -1); vector<vi> g(n + m);
    rep(i, 0, k) {
        int u, v; cin >> u >> v;
        g[u-1].push_back(v + n - 1); // Map right side to [n, n+m)
    }
    int matchingSize = hopcroftKarp(g, r);
    // r[i] stores the left-side match for right-side node i
    for (int i = n; i < n + m; i++)
        if (r[i] != -1) cout << r[i] + 1 << " " << i - n + 1 << endl;
}
```

7 String

7.1 Trie

```
struct trienode {
  bool endmark;
  vector<int> child;
  trienode(int sz) {
    endmark = false;
    child.resize(sz, 0);
  }
};
struct trie {
  int sz;
  char fst;
  vector<trienode> nodes;
  trie(int alpha_sz, char alpha_start) {
    sz = alpha_sz;
    fst = alpha_start;
    // root at idx 0
    nodes.pb(trienode(sz));
  }
  void insert(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst]) {
        nodes[cur].child[c - fst] = nodes.size();
        nodes.pb(trienode(sz));
      }
      cur = nodes[cur].child[c - fst];
    }
    nodes[cur].endmark = true;
  }
  bool search(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst]) return false;
      cur = nodes[cur].child[c - fst];
    }
    return nodes[cur].endmark;
  }
  bool erase(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst]) return false;
      cur = nodes[cur].child[c - fst];
    }
    if (!nodes[cur].endmark) return false;
    nodes[cur].endmark = false;
    return true;
  }
};
```

7.2 Bit Trie

```
#define MAXN 100000011
int trie[MAXN][2], triesz[MAXN], nodes = 0, bitsz = 30;
void insert(int x) {
  int cur = 0;
  for (int i = bitsz; i >= 0; --i) {
    int child = x & (1 << i) ? 1 : 0;
    if (!trie[cur][child]) trie[cur][child] = ++nodes;
    triesz[cur]++;
    cur = trie[cur][child];
  }
  triesz[cur]++;
}
void erase(int x) {
  int cur = 0;
  for (int i = bitsz; i >= 0; --i) {
    int child = x & (1 << i) ? 1 : 0;
    if (!triesz[cur] || !trie[cur][child]) return;
    triesz[cur]--;
    cur = trie[cur][child];
  }
  triesz[cur]--;
}
// max xor with x
int get_max(int x) {
  int cur = 0, ans = 0;
  for (int i = bitsz; i >= 0; --i) {
    int child = x & (1 << i) ? 1 : 0;
    child = 1 - child;  // remove for get_min()
    if (!trie[cur][child] || !triesz[trie[cur][child]]) child = 1 - child;
    cur = trie[cur][child];
    ans |= child << i;
  }
  return ans ^ x;
}
// min xor with x
int get_min(int x) {
  int cur = 0, ans = 0;
  for (int i = bitsz; i >= 0; --i) {
    int child = x & (1 << i) ? 1 : 0;
    // child = 1 - child;  // remove for get_min()
    if (!trie[cur][child] || !triesz[trie[cur][child]]) child = 1 - child;
    cur = trie[cur][child];
    ans |= child << i;
  }
  return ans ^ x;
}
void deleteall(int root) {
  if (trie[root][0]) deleteall(trie[root][0]);
  if (trie[root][1]) deleteall(trie[root][1]);
  trie[root][0] = trie[root][1] = 0;
  triesz[root] = 0;
}
void cleanup() {
  nodes = 0;
  deleteall(0);
}
```

7.3 Z Algorithm

```
VI Z(string& s) {
  int n = s.size();
  VI z(n);
  int l = 0, r = 0;
  for (int i = 0; i < n; ++i) {
    if (i <= r) z[i] = min(z[i - l], r - i + 1);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) l = i, r = i + z[i]++;
  }
  return z;
}
```

7.4 KMP

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -¿ 0010123). Can be used to find all occurrences of a string. Time: O(n)

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g
     -1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
vi match(const string& s, const string&
    pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i
      - 2 * sz(pat));
  return res;
}
```

7.5 Suffix Array

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars. Time: O(nlogn)

```
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string s, int lim=256) {
     // or vector<int>
    s.push_back(0); int n = sz(s), k =
     0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j =
     max(1, j * 2), lim = p) {
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++]
     = sa[i] - j;
      fill(all(ws), 0);
      rep(i,0,n) ws[x[i]]++;
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[
     i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i
     ], x[b] =
        (y[a] == y[b] && y[a + j] == y[b
     + j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[
     i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

7.6 Manacher

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down). Time: O(N)

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n
    ; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t])
     ;
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[
     R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

7.7 Aho Corasick

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. Time: construction takes $O(26N)$, where N = sum of length of patterns. find(x) is $O(N)$, where N = length of x. findAll is $O(NM)$

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; //
     change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1,
     end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof
     (next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = sz(N); N.
     emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N
     (1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);
    queue<int> q;
    for (q.push(0); !q.empty(); q.pop())
     {
      int n = q.front(), prev = N[n].
     back;
      rep(i,0,alpha) {
        int &ed = N[n].next[i], y = N[
     prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end :
      backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].
     nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat
     , string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].
     push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

7.8 Hashing

```
const ll a1 = 911382, b1 = 1e9 + 7;
const ll a2 = 972663, b2 = 1e9 + 9;
ll p1[maxN], p2[maxN];
void pre() {
    p1[0] = p2[0] = 1;
    for (int i = 1; i < maxN; i++) {
        p1[i] = (p1[i - 1] * a1) % b1;
        p2[i] = (p2[i - 1] * a2) % b2;
    }
}
struct Hash {
    vector<ll> h1, h2;
    int n;
    string s;
    Hash(string s) : s(s), n(s.size()) {
        h1.resize(n + 1, 0); h2.resize(n
     + 1, 0);
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * a1 + s[
     i]) % b1;
            h2[i + 1] = (h2[i] * a2 + s[
     i]) % b2;
        }
    }
    pll get(int l, int r) { // [l, r] 0-
     indexed
        ll r1 = (h1[r + 1] - h1[l] * p1[
     r - l + 1]) % b1;
        ll r2 = (h2[r + 1] - h2[l] * p2[
     r - l + 1]) % b2;
        return {r1 < 0 ? r1 + b1 : r1,
     r2 < 0 ? r2 + b2 : r2};
    }
    int count(string& pat) {
        int m = pat.size(), cnt = 0;
        if (m > n) return 0;
        Hash target(pat);
        return count(target, 0, target.n
     -1);
    }
    int search(string& pat) {
        int m = pat.size();
        if (m > n) return -1;
        Hash target(pat);
        return search(target, 0, target.
     n-1);
    }
    int count(Hash& h, int l, int r) {
        int m = r - l + 1, cnt = 0;
        if (m > n) return 0;
        pll target = h.get(l, r);
        for (int i = 0; i <= n - m; i++)
            if (get(i, i + m - 1) ==
     target) cnt++;
        return cnt;
    }
    int search(Hash& h, int l, int r) {
        int m = r - l + 1;
        if (m > n) return -1;
        pll target = h.get(l, r);
        for (int i = 0; i <= n - m; i++)
            if (get(i, i + m - 1) ==
     target) return i;
        return -1;
    }
    int lcp(int i, int j) { // LCP of
     suffix i and suffix j
       int lo = 0, hi = min(n - i, n - j
     );
       while (lo < hi) {
           int mid = (lo + hi + 1) / 2;
           if (get(i, i+mid-1) == get(j,
      j+mid-1)) lo = mid;
           else hi = mid - 1;
       }
       return lo;
   }
   // lcp between this[i,...] amd b[j
     ,...]
   int lcp(Hash& b, int i, int j) {
```

```
        int lo = 0, hi = min(n - i, b.n - j);
        while (lo < hi) {
            int mid = (lo + hi + 1) / 2;
            if (get(i, i+mid-1) == b.get(j, j+mid-1)) lo = mid;
            else hi = mid - 1;
        }
        return lo;
    }
    // LCS of s[..i] and s[..j]
    int lcs(int i, int j) {
        int lo = 0, hi = min(i + 1, j + 1);
        while (lo < hi) {
            int mid = (lo + hi + 1) / 2;
            if (get(i-mid+1, i) == get(j-mid+1, j)) lo = mid;
            else hi = mid - 1;
        }
        return lo;
    }
    // Cross-string LCS: LCS of this[...i] and b[...j]
    int lcs(Hash& b, int i, int j) {
        int lo = 0, hi = min(i + 1, j + 1);
        while (lo < hi) {
            int mid = (lo + hi + 1) / 2;
            if (get(i-mid+1, i) == b.get(j-mid+1, j)) lo = mid;
            else hi = mid - 1;
        }
        return lo;
    }
    // if s+s+s... (1 or more times) = t -> true
    bool repeat(string& s_in) {
        int m = s_in.size();
        if (m == 0 || n % m != 0 || n == 0) return false;
        Hash temp(s_in);
        if (get(0, m - 1) != temp.get(0, m - 1)) return false;
        if (n > m && get(0, n - m - 1) != get(m, n - 1)) return false;
        return true;
    }
};
```

8 Numerical

8.1 FFT

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x]\exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod. Time: $O(N \log N)$ with $N = |A| + |B|$ ($\tilde{1}s$ for $N = 2^{22}$)

```
using C = complex<double>;
using vd = vector<double>;
void fft(vector<C>& a) {
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);
    static int kBuilt = 2;
    for (; kBuilt < n; kBuilt *= 2) {
        R.resize(2 * kBuilt);
        rt.resize(2 * kBuilt);
        auto x = polar(1.0L, acos(-1.0L) / kBuilt);
        for (int i = kBuilt; i < 2 * kBuilt; i++)
            rt[i] = R[i] = (i & 1) ? R[i / 2] * x : R[i / 2];
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                // Hand-rolled complex multiply (~25% faster than rt[j+k]*a[i+j+k])
                auto x = (double*)&rt[j + k];
                auto y = (double*)&a[i + j + k];
                C z(x[0] * y[0] - x[1] * y[1],
                    x[0] * y[1] + x[1] * y[0]);
                a[i + j + k] = a[i + j] - z;
                a[i + j]    += z;
            }
}
vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(a.size() + b.size() - 1);
    int L = 32 - __builtin_clz((int)res.size()), n = 1 << L;
    vector<C> in(n), out(n);
    copy(a.begin(), a.end(), in.begin());
    for (int i = 0; i < (int)b.size(); i++) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    for (int i = 0; i < n; i++) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    for (int i = 0; i < (int)res.size(); i++) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

8.2 FFT With mod M

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $O(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

```
template <ll M>
vector<ll> convMod(vector<ll>& a, vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    vector<ll> res(a.size() + b.size() - 1);
    int B = 32 - __builtin_clz((int)res.size()), n = 1 << B;
    int cut = (int)sqrt((double)M);
    vector<C> fa(n), fb(n), outs(n), outl(n);
    for (int i = 0; i < (int)a.size(); i++) fa[i] = C((int)a[i] / cut, (int)a[i] % cut);
    for (int i = 0; i < (int)b.size(); i++) fb[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(fa); fft(fb);
    for (int i = 0; i < n; i++) {
        int j = -i & (n - 1);
        outl[j] = (fa[i] + conj(fa[j])) * fb[i] / (2.0 * n);
        outs[j] = (fa[i] - conj(fa[j])) * fb[i] / (2.0 * n) / C(0, 1);
    }
    fft(outl); fft(outs);
    for (int i = 0; i < (int)res.size(); i++) {
        ll av = llround(real(outl[i]));
        ll bv = llround(imag(outl[i])) + llround(real(outs[i]));
        ll cv = llround(imag(outs[i]));
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

8.3 FWHT

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two. Time: $O(N \log N)$

```
// Fast Subset Transform
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
     int &u = a[j], &v = a[j + step];
     tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        // inv ? pii(v, u - v) : pii(u + v, u); // OR /// include-line
        // pii(u + v, u - v);
             // XOR /// include-line
    }
  }
  // if (inv) for (int& x : a) x /= sz(a); // XOR only /// include-line
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

9 Matrix

9.1 The Matrix Class

```
struct Mat {
  int n, m;
  vector<vector<int>> a;
  Mat() { }
  Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
  Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0; a = v; }
  inline void make_unit() {
    assert(n == m);
    for (int i = 0; i < n; i++)  {
      for (int j = 0; j < n; j++) a[i][j] = i == j;
    }
  }
  inline Mat operator + (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
      }
    }
    return ans;
  }
  inline Mat operator - (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
      }
    }
    return ans;
  }
  inline Mat operator * (const Mat &b) {
    assert(m == b.n);
    Mat ans = Mat(n, b.m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < b.m; j++) {
        for(int k = 0; k < m; k++) {
          ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] % mod) % mod;
        }
      }
```

```
    }
    return ans;
  }
  inline Mat pow(long long k) {
    assert(n == m);
    Mat ans(n, n), t = a; ans.make_unit();
    while (k) {
      if (k & 1) ans = ans * t;
      t = t * t;
      k >>= 1;
    }
    return ans;
  }
  inline Mat& operator += (const Mat& b) { return *this = (*this) + b; }
  inline Mat& operator -= (const Mat& b) { return *this = (*this) - b; }
  inline Mat& operator *= (const Mat& b) { return *this = (*this) * b; }
  inline bool operator == (const Mat& b) { return a == b.a; }
  inline bool operator != (const Mat& b) { return a != b.a; }
};
```

9.2 Gaussian Elimination

Description: Input : Mat a of size n x (m+1), where the last column is the RHS (b vector) i.e. a represents the augmented matrix [A — b] Output: Fills ‘ans‘ with solution vector x such that A*x = b (mod p) Returns -1 → no solution exists Returns 0 → unique solution Returns k → infinitely many solutions with k free variables

Usage:

```
Mat aug(n, m + 1); fill aug.a[i][j] with coefficients, aug.a[i][m] with RHS vector<int> sol; int result = Gauss(aug, sol); if (result == -1) // no solution else if (result == 0) // unique solution in sol else // result = number of free variables
```

```
int Gauss(Mat a, vector<int> &ans) {
  int n = a.n, m = a.m - 1;
  vector<int> pos(m, -1);
  int free_var = 0;
  const long long MODSQ = (long long)mod * mod;
  int rank = 0;
  for (int col = 0, row = 0; col < m && row < n; col++) {
    int mx = row;
    for (int k = row; k < n; k++) if (a.a[k][col] > a.a[mx][col]) mx = k;
    if (a.a[mx][col] == 0) continue;
    for (int j = col; j <= m; j++) swap(a.a[mx][j], a.a[row][j]);
    pos[col] = row;
    int inv = power(a.a[row][col], mod - 2);
    for (int i = 0; i < n && inv; i++) {
      if (i != row && a.a[i][col]) {
        int x = (1LL * a.a[i][col] * inv) % mod;
        for (int j = col; j <= m && x; j++) {
          if (a.a[row][j]) a.a[i][j] = (MODSQ + a.a[i][j] - 1LL * a.a[row][j] * x) % mod;
        }
      }
    }
    row++; ++rank;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; i++) {
    if (pos[i] == -1) free_var++;
    else ans[i] = (1LL * a.a[pos[i]][m] * power(a.a[pos[i]][i], mod - 2)) % mod;
  }
  for (int i = 0; i < n; i++) {
    long long val = 0;
    for (int j = 0; j < m; j++) val = (val + 1LL * ans[j] * a.a[i][j]) % mod;
    if (val != a.a[i][m]) return -1;
  }
  return free_var;
}
```

9.3 Determinant

Description: Determinant of a matrix.

Usage:

```
Mat a(n, n); // fill a.a[i][j] int d = Det(a); // d = det(A) mod p
```

```
int Det(Mat a) {
  assert(a.n == a.m);
  int n = a.n;
  const long long MODSQ = (long long)mod * mod;
  int det = 1;
  for (int col = 0, row = 0; col < n && row < n; col++) {
    int mx = row;
    for (int k = row; k < n; k++) if (a.a[k][col] > a.a[mx][col]) mx = k;
    if (a.a[mx][col] == 0) { det = 0; continue; }
    for (int j = col; j < n; j++) swap(a.a[mx][j], a.a[row][j]);
    if (row != mx) det = (det == 0 ? 0 : mod - det);
    det = 1LL * det * a.a[row][col] % mod;
    int inv = power(a.a[row][col], mod - 2);
    for (int i = 0; i < n && inv; i++) {
      if (i != row && a.a[i][col]) {
        int x = (1LL * a.a[i][col] * inv) % mod;
        for (int j = col; j < n && x; j++) {
          if (a.a[row][j]) a.a[i][j] = (MODSQ + a.a[i][j] - 1LL * a.a[row][j] * x) % mod;
        }
      }
    }
    row++;
  }
  return det;
}
```

9.4 Matrix Inverse

Description: Inverse of a matrix

Usage:

```
Mat a(n, n), inv; // fill a.a[i][j] if (Inv(a, inv)) { /* use inv */ } else { /* singular */ }
```

```
bool Inv(Mat a, Mat &ans) {
  assert(a.n == a.m);
  int n = a.n;
  vector<int> col(n);
  Mat tmp(n, n); tmp.make_unit();
  for (int i = 0; i < n; i++) col[i] = i;
  for (int i = 0; i < n; i++) {
    int r = i, c = i;
    for (int j = i; j < n; j++)
      for (int k = i; k < n; k++)
        if (a.a[j][k]) { r = j; c = k; goto found; }
    return false;
    found:
    a.a[i].swap(a.a[r]); tmp.a[i].swap(tmp.a[r]);
    for (int j = 0; j < n; j++) { swap(a.a[j][i], a.a[j][c]); swap(tmp.a[j][i], tmp.a[j][c]); }
    swap(col[i], col[c]);
    int v = power(a.a[i][i], mod - 2);
    for (int j = i + 1; j < n; j++) {
      int f = 1LL * a.a[j][i] * v % mod;
      a.a[j][i] = 0;
      for (int k = i + 1; k < n; k++) {
        a.a[j][k] -= 1LL * f * a.a[i][k] % mod;
        if (a.a[j][k] < 0) a.a[j][k] += mod;
      }
      for (int k = 0; k < n; k++) {
        tmp.a[j][k] -= 1LL * f * tmp.a[i][k] % mod;
        if (tmp.a[j][k] < 0) tmp.a[j][k] += mod;
      }
    }
    for (int j = i + 1; j < n; j++) a.a[i][j] = 1LL * a.a[i][j] * v % mod;
    for (int j = 0; j < n; j++) tmp.a[i][j] = 1LL * tmp.a[i][j] * v % mod;
    a.a[i][i] = 1;
  }
  for (int i = n - 1; i > 0; --i) {
    for (int j = 0; j < i; j++) {
      int v = a.a[j][i];
      for (int k = 0; k < n; k++) {
        tmp.a[j][k] -= 1LL * v * tmp.a[i][k] % mod;
        if (tmp.a[j][k] < 0) tmp.a[j][k] += mod;
      }
    }
  }
  ans = Mat(n, n);
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      ans.a[col[i]][col[j]] = tmp.a[i][j];
  return true;
}
```

10 Geometry

10.1 Geometry Primitives

```
#define EPS 1e-12
#define pi 3.14159265358979311599796346854418516159057617187 5L
bool feq(const long double& a, const long double& b) {
  long double del = fabsl(a - b);
  if (del <= EPS) return true;
  return del <= EPS * max(fabsl(a), fabsl(b));
}
```

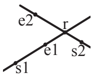
10.2 Point 2D

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T>
int sgn(T x) {
  return (x > 0) - (x < 0);
}
template <class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
  bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
  P operator+(P p) const { return P(x + p.x, y + p.y); }
  P operator-(P p) const { return P(x - p.x, y - p.y); }
  P operator*(T d) const { return P(x * d, y * d); }
  P operator/(T d) const { return P(x / d, y / d); }
  T dot(P p) const { return x * p.x + y * p.y; }
  T cross(P p) const { return x * p.y - y * p.x; }
  T cross(P a, P b) const { return (a - *this).cross(b - *this); }
  T dist2() const { return x * x + y * y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this / dist(); }  // makes dist()=1
  P perp() const { return P(-y, x); }  // rotates +90 degrees
  P normal() const { return perp().unit(); }
```

```
  // returns point rotated 'a' radians
     ccw around the origin
  P rotate(double a) const {
    return P(x * cos(a) - y * sin(a), x
      * sin(a) + y * cos(a));
  }
  friend ostream& operator<<(ostream& os
      , P p) {
    return os << "(" << p.x << "," << p.
      y << ")";
  }
};
```

10.3 Line Intersection

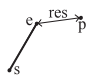


Description: If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point¡ll¿ and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage:
`auto res = lineInter(s1,e1,s2,e2); if (res.first == 1) cout << "intersection point at " << res.second << endl;`

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2,
      P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0)  // if parallel
    return {-(s1.cross(e1, s2) == 0), P
      (0, 0)};
  auto p = s2.cross(e1, e2), q = s2.
    cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

10.4 Segment Distance

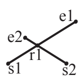


Description: Returns the shortest distance between point p and the line segment from point s to e.

Usage:
`Point<double> a, b(2,2), p(1,1); bool onSegment = segDist(a,b,p) < 1e-10;`

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s == e) return (p - s).dist();
  auto d = (e - s).dist2(), t = min(d,
    max(.0, (p - s).dot(e - s)));
  return ((p - s) * d - (e - s) * t).
    dist() / d;
}
```

10.5 Segment Intersection



Description: If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point¡ll¿ and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage:
`vector<P> inter = segInter(s1,e1,s2,e2); if (sz(inter)==1) cout << "segments intersect at " << inter[0] << endl;`

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(
    d, b), oc = a.cross(b, c),
       od = a.cross(b, d);
  // Checks if intersection is single
    non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) *
      sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa
      )};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

10.6 Side Of

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. P is supposed to be Point¡T¿ where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage:
`bool left = sideOf(p1,p2,q)==1;`

```
template <class P>
int sideOf(P s, P e, P p) {
  return sgn(s.cross(e, p));
}
template <class P>
int sideOf(const P& s, const P& e, const
     P& p, double eps) {
  auto a = (e - s).cross(p - s);
  double l = (e - s).dist() * eps;
  return (a > l) - (a < -l);
}
```

10.7 On Segment

Description: Returns true iff p lies on the line segment from s to e.

Usage:
`\texttt {(segDist(s,e,p)<=epsilon)} instead when using Point<double>.`

```
template <class P>
bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).
    dot(e - p) <= 0;
}
```

10.8 Linear Transformation

Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
typedef Point<double> P;
P linearTransformation(const P& p0,
    const P& p1, const P& q0, const P&
    q1,
                       const P& r) {
  P dp = p1 - p0, dq = q1 - q0, num(dp.
    cross(dq), dp.dot(dq));
  return q0 + P((r - p0).cross(num), (r
    - p0).dot(num)) / dp.dist2();
}
```

10.9 Angle class

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage:
`vector<Angle> v = {w[0], w[0].t360() ...}; // sorted int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; } // sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i`

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t = 0) : x(x),
     y(y), t(t) {}
  Angle operator-(Angle b) const {
    return {x - b.x, y - b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t +
      (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t
      + half()}; }
  Angle t360() const { return {x, y, t +
      1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also
      compare distances
  return make_tuple(a.t, a.half(), a.y *
      (ll)b.x) <
         make_tuple(b.t, b.half(), a.x *
      (ll)b.y);
}
// Given two points, this calculates the
      smallest angle between
// them, i.e., the angle that covers the
      defined line segment.
pair<Angle, Angle> segmentAngles(Angle a
     , Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ? make_pair(a, b)
      : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) {  //
    point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) {  //
    angle b - angle a
  int tu = b.t - a.t;
  a.t = b.t;
  return {a.x * b.x + a.y * b.y, a.x * b
     .y - a.y * b.x, tu - (b < a)};
}
```

10.10 Circle Tangents

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. $O(n)$

```
template <class P>
vector<pair<P, P>> tangents(P c1, double
      r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(),
    h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0) return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2)
     * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v *
      r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

10.11 Minimum Enclosing Circle

Description: Computes the minimum circle that encloses a set of points. $O(n)$

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i, 0, sz(ps)) if ((o - ps[i]).dist
     () > r * EPS) {
    o = ps[i], r = 0;
    rep(j, 0, i) if ((o - ps[j]).dist()
     > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k, 0, j) if ((o - ps[k]).dist
     () > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k
     ]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

10.12 Inside Polygon

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow. $O(n)$

Usage:

```
vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool
in = inPolygon(v, P{3, 3}, false);
```

```
template <class P>
bool inPolygon(vector<P>& p, P a, bool
    strict = true) {
  int cnt = 0, n = sz(p);
  rep(i, 0, n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !
     strict;
    // or: if (segDist(p[i], q, a) <=
     eps) return !strict;
    cnt ^= ((a.y < p[i].y) - (a.y < q.y)
     ) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

10.13 Polygon Area

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
template <class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i, 0, sz(v) - 1) a += v[i].cross(v
     [i + 1]);
  return a;
}
```

10.14 Polygon Center

Description: Returns the center of mass for a polygon. Time: $O(n)$

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0);
  double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(
     v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].
     cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

10.15 Convex Hull

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. $O(nlogn)$

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts) + 1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t,
     reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t - 2].
     cross(h[t - 1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t
     == 2 && h[0] == h[1])};
}
```

10.16 Hull Diameter

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). $O(n)$

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S
     [0]}});
  rep(i, 0, j) for (;; j = (j + 1) % n)
    {
    res = max(res, {(S[i] - S[j]).dist2
     (), {S[i], S[j]}});
    if ((S[(j + 1) % n] - S[j]).cross(S[
     i + 1] - S[i]) >= 0) break;
  }
  return res.second;
}
```

10.17 Point Inside Hull

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included. $O(logN)$

```
typedef Point<ll> P;
bool inHull(const vector<P>& l, P p,
    bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l
    [0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap
    (a, b);
  if (sideOf(l[0], l[a], p) >= r ||
    sideOf(l[0], l[b], p) <= -r) return
     false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a)
     = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

10.18 Line Hull Intersection

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:

- $(-1, -1)$ if no collision,
- $(i, -1)$ if touching the corner i,
- (i, i) if along side $(i, i+1)$,
- (i, j) if crossing sides $(i, i+1)$ and $(j, j+1)$.

In the last case, if a corner i is crossed, this is treated as happening on side $(i, i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line. $O(logN)$

```
#define cmp(i, j) sgn(dir.perp().cross(
    poly[(i) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 &&
    cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m
     + 1, m);
    (ls < ms || (ls == ms && ls == cmp(
     lo, m)) ? hi : lo) = m;
  }
  return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<
    P>& poly) {
  int endA = extrVertex(poly, (a - b).
    perp());
  int endB = extrVertex(poly, (b - a).
    perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i, 0, 2) {
    int lo = endB, hi = endA, n = sz(
    poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 :
     n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi)
     = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0],
     -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly)
     + 1) % sz(poly)) {
      case 0:
        return {res[0], res[0]};
      case 2:
        return {res[1], res[1]};
    }
  return res;
}
```

10.19 Closest Pair

Description: Finds the closest pair of points. O(nlogn)

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y
      < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P
    (), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(
     v[j++]);
    auto lo = S.lower_bound(p - d), hi =
      S.upper_bound(p + d);
    for (; lo != hi; ++lo) ret = min(ret
     , {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

10.20 Polyhedron Volume

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

```
template <class V, class L>
double signedPolyVolume(const V& p,
    const L& trilist) {
  double v = 0;
  for (auto i : trilist) v += p[i.a].
    cross(p[i.b]).dot(p[i.c]);
  return v / 6;
}
```

10.21 Point 3D

Description: lass to handle points in 3D space. T can be e.g. double or long long.

```
template <class T>
struct Point3D {
  typedef Point3D P;
  typedef const P& R;
  T x, y, z;
  explicit Point3D(T x = 0, T y = 0, T z
      = 0) : x(x), y(y), z(z) {}
  bool operator<(R p) const { return tie
    (x, y, z) < tie(p.x, p.y, p.z); }
  bool operator==(R p) const { return
    tie(x, y, z) == tie(p.x, p.y, p.z);
     }
  P operator+(R p) const { return P(x +
    p.x, y + p.y, z + p.z); }
  P operator-(R p) const { return P(x -
    p.x, y - p.y, z - p.z); }
  P operator*(T d) const { return P(x *
    d, y * d, z * d); }
  P operator/(T d) const { return P(x /
    d, y / d, z / d); }
  T dot(R p) const { return x * p.x + y
    * p.y + z * p.z; }
  P cross(R p) const {
    return P(y * p.z - z * p.y, z * p.x
    - x * p.z, x * p.y - y * p.x);
  }
```

```
  T dist2() const { return x * x + y * y + z * z; }
  double dist() const { return sqrt((double)dist2()); }
  // Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
  double phi() const { return atan2(y, x); }
  // Zenith angle (latitude) to the z-axis in interval [0, pi]
  double theta() const { return atan2(sqrt(x * x + y * y), z); }
  P unit() const { return *this / (T)dist(); }  // makes dist()=1
  // returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  // returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle);
    P u = axis.unit();
    return u * dot(u) * (1 - c) + (*this) * c - cross(u) * s;
  }
};
```

10.22 3D Convex Hull

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards. Time: $O(n^2)$

```
struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
};
struct F {
  P3 q;
  int a, b, c;
};
vector<F> hull3d(const vector<P3>& A) {
  assert(sz(A) >= 4);
  vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x, y) E[f.x][f.y]
  vector<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i])) q = q * -1;
    F f{q, i, j, k};
    E(a, b).ins(k);
    E(a, c).ins(j);
    E(b, c).ins(i);
    FS.push_back(f);
  };
  rep(i, 0, 4) rep(j, i + 1, 4) rep(k, j + 1, 4) mf(i, j, k, 6 - i - j - k);
  rep(i, 4, sz(A)) {
    rep(j, 0, sz(FS)) {
      F f = FS[j];
      if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a, b).rem(f.c);
        E(a, c).rem(f.b);
        E(b, c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();
      }
    }
    int nw = sz(FS);
    rep(j, 0, nw) {
      F f = FS[j];
#define C(a, b, c) \
  if (E(a, b).cnt() != 2) mf(f.a, f.b, i, f.c);
      C(a, b, c);
      C(a, c, b);
      C(b, c, a);
    }
  }
  for (F& it : FS)
    if ((A[it.b] - A[it.a]).cross(A[it.c] - A[it.a]).dot(it.q) <= 0)
      swap(it.c, it.b);
  return FS;
};
```

10.23 Spherical Distance

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

```
double sphericalDistance(double f1, double t1, double f2, double t2,
                         double radius) {
  double dx = sin(t2) * cos(f2) - sin(t1) * cos(f1);
  double dy = sin(t2) * sin(f2) - sin(t1) * sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx * dx + dy * dy + dz * dz);
  return radius * 2 * asin(d / 2);
}
```

11 Miscellaneous

11.1 Interval Container

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). Time: O(logN)

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}
void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

11.2 Generationg Permutations

```
VI a;
//...
sort(all(a));
do {
  // Process permutation...
} while (next_permutation(all(a)));
```

11.3 Iterating over Submasks

```
int n = 17;
for (int mask = 0; mask < (1 << n); ++mask) {
  for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // Process submask...
  }
}
```

11.4 Random

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
int randint(int l, int r) {
  uniform_int_distribution<int> dist(l, r);
  return dist(rng);
}
```

11.5 Index compression

```
struct compress {
  vector<int> a;
  compress(vector<int>& v) : a(v) {
    sort(all(a));
    a.erase(unique(all(a)), a.end());
  }
  int size() { return a.size(); }
  int operator[](int val) { return lower_bound(all(a), val) - a.begin(); }
};
```

11.6 pbds

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
// ordered set
ordered_set<int> st;
// ordered multiset
ordered_set<pair<int, int>> multi;
// get index/ number of smaller elements than x
st.order_of_key(x);
// access by index
st.find_by_order(idx);
```

11.7 Bitset

```
bitset<N> b; // All 0s; b[i] access; b.test(i) bounds check
bitset<N> b(string("110")); // b[0]=0, b[1]=1, b[2]=1
b.set(i, v=1); b.reset(i); b.flip(i); // Single bit modify
b.set(); b.reset(); b.flip(); // All bits modify
b.count(); b.any(); b.all(); // popcount; if 1+ bits set; if all bits set
b >>= k; b <<= k; b &= b2; b |= b2; b ^= b2; b = ~b; // O(N/64) bitwise
// GCC Specific (The good stuff):
b._Find_first(); // Index of first '1' (returns N if none)
b._Find_next(i); // Index of next '1' after i (returns N if none)
```

11.8 Digit Range Sum

Description: Gives the digit sum of the numbers in the range [l,r]. I have encountered this problem twice.

```
auto qrsm = [](ll n) -> ll {
  ll ans = 0;
  ll po = 1;
  ll nm = n;
  while (n) {
    ll tm = n / 10;
    ll rem = n % 10;
    ll s1 = rem * (rem - 1) / 2;
    ll past = (nm % po) + 1ll;
    ans += (tm * 45ll) * po + (s1) * (po) + (rem * past);
    po *= 10;
    n /= 10;
  }
  return ans;
};
```

```
auto rsm = [&](ll l, ll r) -> ll {
    return qrsm(r) - qrsm(l - 1); };
```

12 Formulae

12.0.1 Inclusion-Exclusion Principle

Let S_j be the sum of sizes of all intersections of exactly j properties:

$$S_j = \sum_{1 \le i_1 < \cdots < i_j \le n} |A_{i_1} \cap \cdots \cap A_{i_j}|$$

- **Union (At least 1):** $|A_1 \cup \cdots \cup A_n| = \sum_{j=1}^{n} (-1)^{j-1} S_j$
- **Exactly k properties:** $E_k = \sum_{j=k}^{n} (-1)^{j-k} \binom{j}{k} S_j$
- **At least k properties:** $L_k = \sum_{j=k}^{n} (-1)^{j-k} \binom{j-1}{k-1} S_j$

Implementation Note: If intersections depend only on the number of sets j, precompute $S_j = \binom{n}{j} \times \text{intersect}(j)$ in $O(n)$. Otherwise, use bitmask PIE in $O(2^n)$.

12.0.2 Combinatorics

1. $\sum_{0 \le k \le n} \binom{n-k}{k} = Fib_{n+1}$
2. $\binom{n}{k} = \binom{n}{n-k}$
3. $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
4. $k\binom{n}{k} = n\binom{n-1}{k-1}$
5. $\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$
6. $\sum_{i=0}^{n} \binom{n}{i} = 2^n$
7. $\sum_{i \ge 0} \binom{n}{2i} = 2^{n-1}$
8. $\sum_{i \ge 0} \binom{n}{2i+1} = 2^{n-1}$
9. $\sum_{i=0}^{k} (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
10. $\sum_{i=0}^{k} \binom{n+i}{i} = \sum_{i=0}^{k} \binom{n+i}{n} = \binom{n+k+1}{k}$
11. $1\binom{n}{1} + 2\binom{n}{2} + 3\binom{n}{3} + \cdots + n\binom{n}{n} = n2^{n-1}$
12. $1^2\binom{n}{1} + 2^2\binom{n}{2} + 3^2\binom{n}{3} + \cdots + n^2\binom{n}{n} = (n+n^2)2^{n-2}$
13. **Vandermonde's Identify:** $\sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$
14. **Hockey-Stick Identify:** $n, r \in N, n > r, \sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$
15. $\sum_{i=0}^{k} \binom{k}{i}^2 = \binom{2k}{k}$
16. $\sum_{k=0}^{n} \binom{n}{k}\binom{n}{n-k} = \binom{2n}{n}$
17. $\sum_{k=q}^{n} \binom{n}{k}\binom{k}{q} = 2^{n-q}\binom{n}{q}$
18. $\sum_{i=0}^{n} k^i \binom{n}{i} = (k+1)^n$
19. $\sum_{i=0}^{n} \binom{2n}{i} = 2^{2n-1} + \frac{1}{2}\binom{2n}{n}$
20. $\sum_{i=1}^{n} \binom{n}{i}\binom{n-1}{i-1} = \binom{2n-1}{n-1}$
21. $\sum_{i=0}^{n} \binom{2n}{i}^2 = \frac{1}{2}\left(\binom{4n}{2n} + \binom{2n}{n}^2\right)$
22. **Highest Power of 2 that divides ${}^{2n}C_n$:** Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x. Let it form a sequence. The n-th value in the sequence (starting from $n = 0$) gives the highest power of 2 that divides ${}^{2n}C_n$.
23. **Pascal Triangle**
    (a) In a row p where p is a prime number, all the terms in that row except the 1s are multiples of p.
    (b) Parity: To count odd terms in row n, convert n to binary. Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x.
    (c) Every entry in row $2^n - 1, n \ge 0$, is odd.
24. An integer $n \ge 2$ is prime if and only if all the intermediate binomial coefficients $\binom{n}{1}, \binom{n}{2}, \ldots, \binom{n}{n-1}$ are divisible by n.
25. **Kummer's Theorem:** For given integers $n \ge m \ge 0$ and a prime number p, the largest power of p dividing $\binom{n}{m}$ is equal to the number of carries when m is added to n-m in base p. For implementation take inspiration from lucas theorem.
26. Number of different binary sequences of length n such that no two 0's are adjacent$=Fib_{n+1}$
27. **Combination with repetition:** Let's say we choose k elements from an n-element set, the order doesn't matter and each element can be chosen more than once. In that case, the number of different combinations is: $\binom{n+k-1}{k}$
28. Number of ways to divide n persons in $\frac{n}{k}$ equal groups i.e. each having size k is
$$\frac{n!}{k!^{\frac{n}{k}}\left(\frac{n}{k}\right)!} = \prod_{n \ge k}^{n-=k} \binom{n-1}{k-1}$$
29. The number non-negative solution of the equation: $x_1 + x_2 + x_3 + \cdots + x_k = n$ is $\binom{n+k-1}{n}$
30. Number of ways to choose n ids from 1 to b such that every id has distance at least k $= \left(\frac{b-(n-1)(k-1)}{n}\right)$
31. $\sum_{i=1,3,5,\ldots}^{i \le n} \binom{n}{i} a^{n-i} b^i = \frac{1}{2}((a+b)^n - (a-b)^n)$
32. $\sum_{i=0}^{n} \frac{\binom{k}{i}}{\binom{n}{i}} = \frac{\binom{n+1}{n-k+1}}{\binom{n}{k}}$
33. **Derangement:** a permutation of the elements of a set, such that no element appears in its original position. Let $d(n)$ be the number of derangements of the identity permutation fo size n.
    $d(n) = (n-1)\big(d(n-1) + d(n-2)\big)$, where $d(0) = 1,\ d(1) = 0$.
34. **Involutions:** permutations such that $p^2 =$ identity permutation. $a_0 = a_1 = 1$ and $a_n = a_{n-1} + (n-1)a_{n-2}$ for $n > 1$.
35. Let $T(n,k)$ be the number of permutations of size n for which all cycles have length $\le k$.
$$T(n,k) = \begin{cases} n! & ; n \le k \\ n \cdot T(n-1,k) & \\ -F(n-1,k) & \\ \cdot T(n-k-1,k) & ; n > k \end{cases}$$
Here $F(n,k) = n \cdot (n-1) \cdot \cdots \cdot (n-k+1)$
36. **Lucas Theorem**
    (a) If p is prime, then $\binom{p^a}{k} \equiv 0 (\bmod\ p)$
    (b) For non-negative integers m and n and a prime p, the following congruence relation holds: $\binom{m}{n} \equiv \prod_{i=0}^{k} \binom{m_i}{n_i} \pmod{p}$, where, $m = m_k p^k + m_{k-1}p^{k-1} + \cdots + m_1 p + m_0$, and $n = n_k p^k + n_{k-1}p^{k-1} + \cdots + n_1 p + n_0$ are the base p expansions of m and n respectively. This uses the convention that $\binom{m}{n} = 0$,when $m < n$.
37. $\sum_{i=0}^{n} \binom{n}{i} \cdot i^k$
$$= \sum_{i=0}^{n} \binom{n}{i} \cdot \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot i^{\underline{j}}$$
$$= \sum_{i=0}^{n} \binom{n}{i} \cdot \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot j! \binom{n}{i}$$
$$= \sum_{i=0}^{n} \frac{n!}{(n-i)!} \cdot \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot \frac{1}{(i-j)!}$$
$$= \sum_{i=0}^{n} \sum_{j=0}^{k} \frac{n!}{(n-i)!} \cdot \left\{ {k \atop j} \right\} \cdot \frac{1}{(i-j)!}$$
$$= \text{n!} \sum_{i=0}^{n} \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot \frac{1}{(n-i)!} \cdot \frac{1}{(i-j)!}$$
$$= \text{n!} \sum_{i=0}^{n} \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot \binom{n-j}{n-i} \cdot \frac{1}{(n-j)!}$$
$$= \text{n!} \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot \frac{1}{(n-j)!} \sum_{i=0}^{n} \cdot \binom{n-j}{n-i}$$
$$= \sum_{j=0}^{k} \left\{ {k \atop j} \right\} \cdot n^{\underline{j}} \cdot 2^{n-j}$$
Here $n^{\underline{j}} = P(n,j) = \frac{n!}{(n-j)!}$ and $\left\{ {k \atop j} \right\}$ is stirling number of the second kind.
So, instead of $O(n)$, now you can calculate the original equation in $O(k^2)$ or even in $O(k \log^2 n)$ using NTT.
38. $\sum_{i=0}^{n-1} \binom{i}{j} x^i = x^j (1-x)^{-j-1} \cdot \left(1 - x^n \sum_{i=0}^{j} \binom{n}{i} x^{j-i} (1-x)^i\right)$
39. $x_0, x_1, x_2, x_3, \ldots, x_n$
    $x_0 + x_1, x_1 + x_2, x_2 + x_3, \ldots x_n$
    ...
    If we continuously do this n times then the polynomial of the first column of the n-th row will be
$$p(n) = \sum_{k=0}^{n} \binom{n}{k} \cdot x(k)$$
40. If $P(n) = \sum_{k=0}^{n} \binom{n}{k} \cdot Q(k)$, then,
$$Q(n) = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} \cdot P(k)$$

41. If $P(n) = \sum_{k=0}^{n} (-1)^k \binom{n}{k} \cdot Q(k)$, then,
$$Q(n) = \sum_{k=0}^{n} (-1)^k \binom{n}{k} \cdot P(k)$$

12.0.3 Catalan Numbers

42. $C_n = \frac{1}{n+1}\binom{2n}{n}$
43. $C_0 = 1, C_1 = 1$ and $C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$
44. Number of correct bracket sequence consisting of n opening and n closing brackets.
45. The number of ways to completely parenthesize $n+1$ factors.
46. The number of triangulations of a convex polygon with $n+2$ sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).
47. The number of ways to connect the $2n$ points on a circle to form n disjoint i.e. non-intersecting chords.
48. The number of monotonic lattice paths from point $(0,0)$ to point (n,n) in a square lattice of size $n \times n$, which do not pass above the main diagonal (i.e. connecting $(0,0)$ to (n,n)).
49. The number of rooted full binary trees with $n+1$ leaves (vertices are not numbered). A rooted binary tree is full if every vertex has either two children or no children.
50. Number of permutations of $\{1,\ldots,n\}$ that avoid the pattern 123 (or any of the other patterns of length 3); that is, the number of permutations with no three-term increasing subsequence. For $n = 3$, these permutations are $132, 213, 231, 312$ and 321. For $n = 4$, they are $1432, 2143, 2413, 2431, 3142, 3214, 3241, 3412, 3421, 4132, 4213, 4231, 4312$ and 4321.
51. **Balanced Parentheses count with prefix:** The count of balanced parentheses sequences consisting of $n+k$ pairs of parentheses where the first k symbols are open brackets. Let the number be $C_n^{(k)}$, then
$$C_n^{(k)} = \frac{k+1}{n+k+1}\binom{2n+k}{n}$$

12.0.4 Narayana numbers

52. $N(n,k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1}$
53. The number of expressions containing n pairs of parentheses, which are correctly matched and which contain k distinct nestings. For instance, $N(4,2) = 6$ as with four pairs of parentheses six sequences can be created which each contain two times the sub-pattern '()'.

12.0.5 Stirling numbers of the first kind

54. The Stirling numbers of the first kind count permutations according to their number of cycles (counting fixed points as cycles of length one).
55. $S(n,k)$ counts the number of permutations of n elements with k disjoint cycles.
56. $S(n,k) = (n-1) \cdot S(n-1,k) + S(n-1,k-1),\ where,\ S(0,0) = 1, S(n,0) = S(0,n) = 0$
57. $\sum_{k=0}^{n} S(n,k) = n!$
58. The unsigned Stirling numbers may also be defined algebraically, as the coefficient of the rising factorial:
$$x^{\bar{n}} = x(x+1)\ldots(x+n-1) = \sum_{k=0}^{n} S(n,k)x^k$$
59. Lets $[n,k]$ be the stirling number of the first kind, then
$$\left[{n \atop n-k}\right] = \sum_{0 \le i_1 < i_2 < i_k < n} i_1 i_2 \cdots{}_k .$$

12.0.6 Stirling numbers of the second kind

60. Stirling number of the second kind is the number of ways to partition a set of n objects into k non-empty subsets.
61. $S(n,k) = k \cdot S(n-1,k) + S(n-1,k-1),\ where\ S(0,0) = 1, S(n,0) = S(0,n) = 0$
62. $S(n,2) = 2^{n-1} - 1$
63. $S(n,k) \cdot k! =$ number of ways to color n nodes using colors from 1 to k such that each color is used at least once.
64. An r-associated Stirling number of the second kind is the number of ways to partition a set of n objects into k subsets, with each subset containing at least r elements. It is denoted by $S_r(n,k)$ and obeys the recurrence relation. $S_r(n+1,k) = kS_r(n,k) + \binom{n}{r-1}S_r(n-r+1,k-1)$
65. Denote the n objects to partition by the integers $1,2,\ldots,n$. Define the reduced Stirling numbers of the second kind, denoted $S^d(n,k)$, to be the number of ways to partition the integers $1,2,\ldots,n$ into k nonempty subsets such that all elements in each subset have pairwise distance at least d. That is, for any integers i and j in a given subset, it is required that $|i-j| \ge d$. It has been shown that these numbers satisfy, $S^d(n,k) = S(n-d+1,k-d+1), n \ge k \ge d$

12.0.7 Bell number

66. Counts the number of partitions of a set.
67. $B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} \cdot B_k$
68. $B_n = \sum_{k=0}^{n} S(n,k)$,where $S(n,k)$ is stirling number of second kind.

12.0.8 Math

69. $ab \mod ac = a(b \mod c)$
70. $\sum_{i=0}^{n} i \cdot i! = (n+1)! - 1.$
71. $a^k - b^k = (a-b) \cdot (a^{k-1}b^0 + a^{k-2}b^1 + \cdots + a^0 b^{k-1})$
72. $\min(a+b,c) = a + \min(b, c-a)$
73. $|a-b| + |b-c| + |c-a| = 2(\max(a,b,c) - \min(a,b,c))$
74. $a \cdot b \le c \to a \le \left\lfloor \frac{c}{b} \right\rfloor$ is correct
75. $a \cdot b < c \to a < \left\lfloor \frac{c}{b} \right\rfloor$ is incorrect
76. $a \cdot b \ge c \to a \ge \left\lfloor \frac{c}{b} \right\rfloor$ is correct
77. $a \cdot b > c \to a > \left\lfloor \frac{c}{b} \right\rfloor$ is correct
78. For positive integer n, and arbitrary real numbers m, x, $\left\lfloor \frac{\lfloor x/m \rfloor}{n} \right\rfloor = \left\lfloor \frac{x}{mn} \right\rfloor$ $\left\lceil \frac{\lceil x/m \rceil}{n} \right\rceil = \left\lceil \frac{x}{mn} \right\rceil$
79. Lagrange's identity:
$$\left(\sum_{k=1}^{n} a_k^2\right)\left(\sum_{k=1}^{n} b_k^2\right) - \left(\sum_{k=1}^{n} a_k b_k\right)^2 = \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} (a_i b_j - a_j b_i)^2 = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1, j\neq i}^{n} (a_i b_j - a_j b_i)^2$$
80. $\sum_{i=1}^{n} ia^i = \frac{a(na^{n+1} - (n+1)a^n + 1)}{(a-1)^2}$
81. **Vieta's formulas:** Any general polynomial of degree n
$$p(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$$
(with the coefficients being real or complex numbers and $a_n \neq 0$) is known by the fundamental theorem of algebra to have n (not necessarily distinct) complex roots $r_1, r_2, \ldots, r_n$.
$$\begin{cases} r_1 + r_2 + \cdots + r_{n-1} + r_n = -\frac{a_{n-1}}{a_n} \\ (r_1 r_2 + r_1 r_3 + \cdots + r_1 r_n) \\ +(r_2 r_3 + r_2 r_4 + \cdots + r_2 r_n) + \ldots \\ +r_{n-1}r_n = \frac{a_{n-2}}{a_n} \\ \vdots \\ r_1 r_2 \ldots r_n = (-1)^n \frac{a_0}{a_n}. \end{cases}$$
Vieta's formulas can equivalently be written as
$$\sum_{1 \le i_1 < i_2 < \cdots < i_k \le n} \left(\prod_{j=1}^{k} r_{i_j}\right) = (-1)^k \frac{a_{n-k}}{a_n},$$
82. We are given n numbers $a_1, a_2, \ldots, a_n$ and our task is to find a value x that minimizes the sum,
$$|a_1 - x| + |a_2 - x| + \cdots + |a_n - x|$$
optimal $x =$ median of the array. if n is even $x =$ [left median,right median] i.e. every number in this range will work. For minimizing
$$(a_1 - x)^2 + (a_2 - x)^2 + \cdots + (a_n - x)^2$$
optimal $x = \frac{(a_1 + a_2 + \cdots + a_n)}{n}$
83. Given an array a of n non-negative integers. The task is to find the sum of the product of elements of all the possible subsets. It is equal to the product of $(a_i + 1)$ for all a_i
84. **Pentagonal number theorem:** In mathematics, the pentagonal number theorem states that
$$\prod_{n=1}^{\infty}(1-x^n) = \sum_{k=-\infty}^{\infty}(-1)^k x^{\frac{k(3k-1)}{2}} = 1 + \sum_{k=1}^{\infty}(-1)^k \left(x^{\frac{k(3k+1)}{2}} + x^{\frac{k(3k-1)}{2}}\right).$$
In other words,
$$(1-x)(1-x^2)(1-x^3)\cdots = 1 - x - x^2 + x^5 + x^7 - x^{12} - x^{15} + x^{22} + x^{26} - \cdots.$$
The exponents $1, 2, 5, 7, 12, \cdots$ on the right hand side are given by the formula $g_k = \frac{k(3k-1)}{2}$ for $k = 1, -1, 2, -2, 3, \cdots$ and are called (generalized) pentagonal numbers. It is useful to find the partition number in $O(n\sqrt{n})$

12.0.9 Fibonacci Number

85. $F_0 = 0, F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$
86. $F_n = \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n-k-1}{k}$
87. $F_n = \frac{1}{\sqrt{5}}(\frac{1+\sqrt{5}}{2})^n - \frac{1}{\sqrt{5}}(\frac{1-\sqrt{5}}{2})^n$
88. $\sum_{i=1}^{n} F_i = F_{n+2} - 1$
89. $\sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$
90. $\sum_{i=1}^{n} F_{2i} = F_{2n+1} - 1$
91. $\sum_{i=1}^{n} F_i^2 = F_n F_{n+1}$
92. $F_m F_{n+1} - F_{m-1}F_n = (-1)^n F_{m-n}$ $F_{2n} = F_{n+1}^2 - F_{n-1}^2 = F_n(F_{n+1} + F_{n-1})$
93. $F_m F_n + F_{m-1}F_{n-1} = F_{m+n-1}$ $F_m F_{n+1} + F_{m-1}F_n = F_{m+n}$
94. A number is Fibonacci if and only if one or both of $(5 \cdot n^2 + 4)$ or $(5 \cdot n^2 - 4)$ is a perfect square
95. Every third number of the sequence is even and more generally, every k^{th} number of the sequence is a multiple of F_k
96. $gcd(F_m, F_n) = F_{gcd(m,n)}$
97. Any three consecutive Fibonacci numbers are pairwise coprime, which means that, for every n, $gcd(F_n, F_{n+1}) = gcd(F_n, F_{n+2}), gcd(F_{n+1}, F_{n+2}) = 1$
98. If the members of the Fibonacci sequence are taken *mod n*, the resulting sequence is periodic with period at most $6n$.

12.0.10 Pythagorean Triples

99. A Pythagorean triple consists of three positive integers a, b, and C, such that $a^2 + b^2 = c^2$. Such a triple is commonly written (a, b, c)
100. Euclid's formula is a fundamental formula for generating Pythagorean triples given an arbitrary pair of integers m and n with $m > n > 0$. The formula states that the integers

$$a = m^2 - n^2, b = 2mn, c = m^2 + n^2$$

form a Pythagorean triple. The triple generated by Euclid's formula is primitive if and only if m and n are coprime and not both odd. When both m and n are odd, then a, b, and c will be even, and the triple will not be primitive; however, dividing a, b, and c by 2 will yield a primitive triple when m and n are coprime and both odd.

101. The following will generate all Pythagorean triples uniquely:

$$a = k\,(m^2 - n^2),$$
$$b = k\,(2mn), c = k\,(m^2 + n^2).$$

where m, n, and k are positive integers with $m > n$, and with m and n coprime and not both odd.

102. **Theorem:** The number of Pythagorean triples a,b,n with $maxa, b, n = n$ is given by

$$\frac{1}{2}\left(\prod_{p^\alpha || n} (2\alpha + 1) - 1\right)$$

where the product is over all prime divisors p of the form $4k + 1$. The notation $p^\alpha || n$ stands for the highest exponent α for which p^α divides n **Example:** For $n = 2 \cdot 3^2 \cdot 5^3 \cdot 7^4 \cdot 11^5 \cdot 13^6$, the number of Pythagorean triples with hypotenuse n is $\frac{1}{2}(7.13 - 1) = 45$. To obtain a formula for the number of Pythagorean triples with hypotenuse less than a specific positive integer N, we may add the numbers corresponding to each $n < N$ given by the Theorem. There is no simple way to compute this as a function of N.

12.0.11 Sum of Squares Function

103. The function is defined as

$$r_k(n) = \left| \begin{matrix} (a_1, a_2, \ldots, a_k) \in \mathbf{Z}^k : \\ n = a_1^2 + a_2^2 + \cdots + a_k^2 \end{matrix} \right|$$

104. The number of ways to write a natural number as sum of two squares is given by $r_2(n)$. It is given explicitly by $r_2\,(n) = 4\,(d_1\,(n) - d_3\,(n))$ where d1(n) is the number of divisors of n which are congruent with 1 modulo 4 and d3(n) is the number of divisors of n which are congruent with 3 modulo 4. The prime factorization $n = 2^g p_1^{f_1} p_2^{f_2} ... q_1^{h_1} q_2^{h_2} ...$, where p_i are the prime factors of the form $p_i \equiv 1 \pmod 4$, and q_i are the prime factors of the form $q_i \equiv 3 \pmod 4$ gives another formula $r_2\,(n) = 4\,(f_1 + 1)\,(f_2 + 1) \ldots$, if all exponents h_1,h_2,... are even. If one or more h_i are odd, then $r_2\,(n) = 0$.
105. The number of ways to represent n as the sum of four squares is eight times the sum of all its divisors which are not divisible by 4, i.e. $r_4\,(n) = 8 \sum_{d|n; 4\nmid d} d$

$$r_8\,(n) = 16 \sum_{d|n} (-1)^{n+d}\,d^3$$

12.0.12 Number Theory

106. for $i > j$, $\gcd(i, j) = \gcd(i-j, j) \leq (i-j)$
107. $\sum_{x=1}^{n} \left[d | x^k\right] = \left\lfloor \frac{n}{\prod_{i=0} p_i^{\left\lceil \frac{e_i}{k} \right\rceil}} \right\rfloor$, where $d = \prod_{i=0} p_i^{e_i}$. Here, $[a|b]$ means if a divides b then it is 1, otherwise it is 0.
108. The number of lattice points on segment (x_1, y_1) to (x_2, y_2) is $\gcd(abs(x_1 - x_2), abs(y_1 - y_2)) + 1$
109. $(n-1)! \mod n = n - 1$ if n is prime, 2 if $n = 4$, 0 otherwise.
110. A number has odd number of divisors if it is perfect square
111. The sum of all divisors of a natural number n is odd if and only if $n = 2^r \cdot k^2$ where r is non-negative and k is positive integer.
112. Let a and b be coprime positive integers, and find integers $a\prime$ and $b\prime$ such that $aa\prime \equiv 1 \mod b$ and $bb\prime \equiv 1 \mod a$. Then the number of representations of a positive integers (n) as a non negative linear combination of a and b is

$$\frac{n}{ab} - \left\{\frac{b\prime n}{a}\right\} - \left\{\frac{a\prime n}{b}\right\} + 1$$

Here, x denotes the fractional part of x.

$$\sum_{i=1}^{a} \sum_{j=1}^{b} \sum_{k=1}^{c} d(i \cdot j \cdot k)$$
$$= \sum_{\gcd(i,j)=\gcd(j,k)=\gcd(k,i)=1} \left\lfloor \frac{a}{i} \right\rfloor \left\lfloor \frac{b}{j} \right\rfloor \left\lfloor \frac{c}{k} \right\rfloor$$

Here, $d(x) =$ number of divisors of x.

113. **Gauss's generalization of Wilson's theorem:**, Gauss proved that,

$$\prod_{\substack{k=1 \\ \gcd(k,m)=1}}^{m} k$$

$$\equiv \begin{cases} -1 \pmod m & \text{if } m = 4,\ p^\alpha,\ 2p^\alpha \\ 1 \pmod m & \text{otherwise} \end{cases}$$

where p represents an odd prime and α a positive integer. The values of m for which the product is -1 are precisely the ones where there is a primitive root modulo m.

12.0.13 Divisor Function

114. $\sigma_x(n) = \sum_{d|n} d^x$
115. It is multiplicative i.e if $\gcd(a, b) = 1 \rightarrow \sigma_x(ab) = \sigma_x(a)\sigma_x(b)$.
116. $$\sigma_x(n) = \prod_{i=1}^{\tau} \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$$
117. **Divisor Summatory Function**
    (a) Let $\sigma_0(k)$ be the number of divisors of k.
    (b) $D(x) = \sum_{n \leq x} \sigma_0(n)$
    (c) $D(x) = \sum_{k=1}^{x} \lfloor \frac{x}{k} \rfloor = 2 \sum_{k=1}^{u} \lfloor \frac{x}{k} \rfloor - u^2$, where $u = \sqrt{x}$
    (d) $D(n)$ =Number of increasing arithmetic progressions where $n + 1$ is the second or later term. (i.e. The last term, starting term can be any positive integer $\leq n$. For example, $D(3) = 5$ and there are 5 such arithmetic progressions: $(1, 2, 3, 4); (2, 3, 4); (1, 4); (2, 4); (3, 4)$.
118. Let $\sigma_1(k)$ be the sum of divisors of k. Then, $\sum_{k=1}^{n} \sigma_1(k) = \sum_{k=1}^{n} k \left\lfloor \frac{n}{k} \right\rfloor$
119. $\prod_{d|n} d = n^{\frac{\sigma_0}{2}}$ if n is not a perfect square, and $= \sqrt{n} \cdot n^{\frac{\sigma_0 - 1}{2}}$ if n is a perfect square.

12.0.14 Euler's Totient function

120. The function is multiplicative. This means that if $\gcd(m, n) = 1$, $\phi(m \cdot n) = \phi(m) \cdot \phi(n)$.
121. $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$
122. If p is prime and $(k \geq 1)$, then: $\phi(p^k) = p^{k-1}(p-1) = p^k(1 - \frac{1}{p})$
123. $J_k(n)$, the Jordan totient function, is the number of k-tuples of positive integers all less than or equal to n that form a coprime $(k+1)$-tuple together with n. It is a generalization of Euler's totient, $\phi(n) = J_1(n)$. $J_k(n) = n^k \prod_{p|n} (1 - \frac{1}{p^k})$
124. $\sum_{d|n} J_k(d) = n^k$
125. $\sum_{d|n} \phi(d) = n$
126. $\phi(n) = \sum_{d|n} \mu(d) \cdot \frac{n}{d} = n \sum_{d|n} \frac{\mu(d)}{d}$
127. $\phi(n) = \sum_{d|n} d \cdot \mu(\frac{n}{d})$
128. $a|b \rightarrow \varphi(a)|\varphi(b)$
129. $n|\varphi(a^n - 1)$ for $a, n > 1$
130. $\varphi(mn) = \varphi(m)\varphi(n) \cdot \frac{d}{\varphi(d)}$ where $d = gcd(m, n)$ Note the special cases

$$\varphi(2m) = \begin{cases} 2\varphi(m) & ; if\, m\, is\, even \\ \varphi(m) & ; if\, m\, is\, odd \end{cases}$$

$$\varphi(n^m) = n^{m-1}\varphi(n)$$

131. $\varphi(lcm(m,n))\cdot\varphi(gcd(m,n)) = \varphi(m)\cdot\varphi(n)$ Compare this to the formula $lcm(m,n)\cdot gcd(m,n) = m\cdot n$
132. $\varphi(n)$ is even for $n \geq 3$. Moreover, if if n has r distinct odd prime factors, $2^r|\varphi(n)$
133. $\sum_{d|n}\frac{\mu^2(d)}{\varphi(d)} = \frac{n}{\varphi(n)}$
134. $\sum_{1\leq k\leq n, gcd(k,n)=1} k = \frac{1}{2}n\varphi(n)$ for $n > 1$
135. $\frac{\varphi(n)}{n} = \frac{\varphi(rad(n))}{rad(n)}$ where $rad(n) = \prod_{p|n, p\, prime} p$
136. $\phi(m) \geq \log_2 m$
137. $\phi(\phi(m)) \leq \frac{m}{2}$
138. When $x \geq \log_2 m$, then
$n^x \mod m = n^{\phi(m)+x \mod \phi(m)} \mod m$
139. $\sum_{1\leq k\leq n, gcd(k,n)=1} \gcd(k-1,n) = \varphi(n)d(n)$ where $d(n)$ is number of divisors. Same equation for $\gcd(a\cdot k-1,n)$ where a and n are coprime.
140. For every n there is at least one other integer $m \neq n$ such that $\varphi(m) = \varphi(n)$.
141. $\sum_{i=1}^{n}\varphi(i)\cdot\lfloor\frac{n}{i}\rfloor = \frac{n*(n+1)}{2}$
142. $\sum_{i=1, i\%2\neq 0}^{n}\varphi(i)\cdot\lfloor\frac{n}{i}\rfloor = \sum_{k\geq 1}[\frac{n}{2^k}]^2$. Note that [] is used here to denote round operator not floor or ceil
143.
$$\sum_{i=1}^{n}\sum_{j=1}^{n} ij[\gcd(i,j)=1] = \sum_{i=1}^{n}\varphi(i)i^2$$
144. Average of coprimes of n which are less than n is $\frac{n}{2}$.

12.0.15 Mobius Function and Inversion

145. It is a multiplicative function.
146.
$$\sum_{d|n}\mu(d) = \begin{cases}1 & ; n=1\\ 0 & ; n>0\end{cases}$$
147. $\sum_{n=1}^{N}\mu^2(n) = \sum_{n=1}^{\sqrt{N}}\mu(k)\cdot\left\lfloor\frac{N}{k^2}\right\rfloor$ This is also the number of square-free numbers $\leq n$
148. **Mobius inversion theorem:** The classic version states that if g and f are arithmetic functions satisfying $g(n) = \sum_{d|n} f(d)$ for every integer $n \geq 1$ then $g(n) = \sum_{d|n}\mu(d)g\left(\frac{n}{d}\right)$ for every integer $n \geq 1$
149. If $F(n) = \prod_{d|n} f(d)$, then $F(n) = \prod_{d|n} F\left(\frac{n}{d}\right)^{\mu(d)}$
150. $\sum_{d|n}\mu(d)\phi(d) = \prod_{j=1}^{K}(2-P_j)$ where p_j is the primes factorization of d
151. If $F(n)$ is multiplicative, $F \not\equiv 0$, then $\sum_{d|n}\mu(d)f(d) = \prod_{i=1}(1-f(P_i))\cdot$ where p_i are primes of n.

12.0.16 GCD and LCM

152. $\gcd(a,0) = a$
153. $\gcd(a,b) = \gcd(b, a \mod b)$
154. Every common divisor of a and b is a divisor of $\gcd(a,b)$.
155. if m is any integer, then $\gcd(a+m\cdot b, b) = \gcd(a,b)$
156. The gcd is a multiplicative function in the following sense: if a_1 and a_2 are relatively prime, then $\gcd(a_1\cdot a_2, b) = \gcd(a_1,b)\cdot\gcd(a_2,b)$.
157. $\gcd(a,b)\cdot \text{lcm}(a,b) = |a\cdot b|$
158. $\gcd(a, \text{lcm}(b,c)) = \text{lcm}(\gcd(a,b), \gcd(a,c))$.
159. $\text{lcm}(a, \gcd(b,c)) = \gcd(\text{lcm}(a,b), \text{lcm}(a,c))$.
160. For non-negative integers a and b, where a and b are not both zero, $\gcd(n^a-1, n^b-1) = n^{\gcd(a,b)}-1$
161. $\gcd(a,b) = \sum_{k|a \text{ and } k|b}\phi(k)$
162. $\sum_{i=1}^{n}[\gcd(i,n)=k] = \phi\left(\frac{n}{k}\right)$
163. $\sum_{k=1}^{n}\gcd(k,n) = \sum_{d|n} d\cdot\phi\left(\frac{n}{d}\right)$
164. $\sum_{k=1}^{n} x^{\gcd(k,n)} = \sum_{d|n} x^d\cdot\phi\left(\frac{n}{d}\right)$
165. $\sum_{k=1}^{n}\frac{1}{\gcd(k,n)} = \sum_{d|n}\frac{1}{d}\cdot\phi\left(\frac{n}{d}\right) = \frac{1}{n}\sum_{d|n} d\cdot\phi(d)$
166. $\sum_{k=1}^{n}\frac{k}{\gcd(k,n)} = \frac{n}{2}\cdot\sum_{d|n}\frac{1}{d}\cdot\phi\left(\frac{n}{d}\right) = \frac{n}{2}\cdot\frac{1}{n}\cdot\sum_{d|n} d\cdot\phi(d)$
167. $\sum_{k=1}^{n}\frac{n}{\gcd(k,n)} = 2*\sum_{k=1}^{n}\frac{k}{\gcd(k,n)} - 1$, for $n > 1$
168. $\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j)=1] = \sum_{d=1}^{n}\mu(d)\lfloor\frac{n}{d}\rfloor^2$
169. $\sum_{i=1}^{n}\sum_{j=1}^{n}\gcd(i,j) = \sum_{d=1}^{n}\phi(d)\lfloor\frac{n}{d}\rfloor^2$
170. $\sum_{i=1}^{n}\sum_{j=1}^{n} i\cdot j[\gcd(i,j)=1] = \sum_{i=1}^{n}\phi(i)i^2$
171. $F(n) = \sum_{i=1}^{n}\sum_{j=1}^{n}\text{lcm}(i,j) = \sum_{l=1}^{n}\left(\frac{\left(1+\lfloor\frac{n}{l}\rfloor\right)\left(\lfloor\frac{n}{l}\rfloor\right)}{2}\right)^2\sum_{d|l}\mu(d)ld$
172. $\gcd(\text{lcm}(a,b), \text{lcm}(b,c), \text{lcm}(a,c)) = \text{lcm}(\gcd(a,b), \gcd(b,c), \gcd(a,c))$
173. $\gcd(A_L, A_{L+1}, \ldots, A_R) = \gcd(A_L, A_{L+1}-A_L, \ldots, A_R - A_{R-1})$.
174. Given n, If $SUM = LCM(1,n) + LCM(2,n) + \cdots + LCM(n,n)$ then SUM $= \frac{n}{2}(\sum_{d|n}(\phi(d)\times d) + 1$

12.0.17 Legendre Symbol

175. $\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod p$ and $\left(\frac{a}{p}\right) \in \{-1,0,1\}$.
176. The Legendre symbol is periodic in its first (or top) argument: if $a \equiv b \pmod p$, then $\left(\frac{a}{p}\right) = \left(\frac{b}{p}\right)$.
177. The Legendre symbol is a completely multiplicative function of its top argument: $\left(\frac{ab}{p}\right) = \left(\frac{a}{p}\right)\left(\frac{b}{p}\right)$
178. The Fibonacci numbers $1,1,2,3,5,8,13\ldots$ are defined by the recurrence $F_1 = F_2 = 1, F_{n+1} = F_n + F_{n-1}$. If p is a prime number then $F_{p-(\frac{p}{5})} \equiv 0 \pmod p$, $F_p \equiv \left(\frac{p}{5}\right) \pmod p$.
179. Continuing from previous point, $\left(\frac{p}{5}\right)$ = infinite concatenation of the sequence $(1,-1,-1,1,0)$ from $p \geq 1$.
180. If $n = k^2$ is perfect square then $\left(\frac{n}{p}\right) = 1$ for every odd prime except $\left(\frac{n}{k}\right) = 0$ if k is an odd prime.

12.0.18 Miscellaneous

181. $a + b = a \oplus b + 2(a\&b)$.
182. $a + b = a \mid b + a\&b$
183. $a \oplus b = a \mid b - a\&b$
184. k_{th} bit is set in x iff $x \mod 2^{k-1} \geq 2^k$. It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.
185. k_{th} bit is set in x iff $x \mod 2^{k-1} - x \mod 2^k \neq 0$ ($= 2^k$ to be exact). It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.
186. $n \mod 2^i = n\&(2^i-1)$
187. $1\oplus 2\oplus 3\oplus\cdots\oplus(4k-1) = 0$ for any $k \geq 0$
188. **Erdos Gallai Theorem:** The degree sequence of an undirected graph is the non-increasing sequence of its vertex degrees A sequence of non-negative integers $d_1 \geq d_2 \geq \cdots \geq d_n$ can be represented as the degree sequence of finite simple graph on n vertices if and only if $d_1 + d_2 + \cdots + d_n$ is even and
$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n}\min(d_i, k)$$
holds for every k in $1 \leq k \leq n$.
189. $f_n = \sum_{i=0}^{n}\binom{n}{i}g_i \Rightarrow g_n = \sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f_i$

12.0.19 Traingle

- **Sine Rule:** $\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$
- **Cosine Rule:** $c^2 = a^2 + b^2 - 2ab\cos C$
- **Area:** $\Delta = \sqrt{s(s-a)(s-b)(s-c)} = \frac{1}{2}bc\sin A$
- **Circumradius (R):** $R = \frac{abc}{4\Delta} = \frac{a}{2\sin A}$
- **Inradius (r):** $r = \frac{\Delta}{s}$
- **Ex-radii (r_a, r_b, r_c):** $r_a = \frac{\Delta}{s-a}$
- **Centroid (G):** $\left(\frac{\sum x}{3}, \frac{\sum y}{3}\right)$
- **Incenter (I):** $\left(\frac{ax_1+bx_2+cx_3}{a+b+c}, \frac{ay_1+by_2+cy_3}{a+b+c}\right)$
- **Excenter (I_A):** Replace a with $-a$ in Incenter formula.
- **Euler Line:** O, G, H are collinear; $HG = 2GO$.
- **Median (m_a):** $4m_a^2 = 2b^2 + 2c^2 - a^2$ (Apollonius)
- **Angle Bisector (l_a):** $l_a = \frac{2bc}{b+c}\cos(A/2)$

- **Stewart's Theorem:** Cevian(a line from A to a) length d dividing a into m, n:

$$b^2m + c^2n = a(d^2 + mn)$$

- **Centroid (G):** Intersection of *medians* (vertex to midpoint).
- **Incenter (I):** Intersection of *angle bisectors.*
- **Circumcenter (O):** Intersection of *perpendicular bisectors* of the sides.
- **Orthocenter (H):** Intersection of *altitudes* (vertex perpendicular to opposite side).

12.0.20 Circle

- **General Eq:** $x^2 + y^2 + 2gx + 2fy + c = 0$. Center $(-g, -f)$, $r = \sqrt{g^2 + f^2 - c}$.
- **Tangent at $P(x_1, y_1)$:** $xx_1 + yy_1 + g(x + x_1) + f(y + y_1) + c = 0$
- **Slope of tangent from $P(x, y)$:** $y + f = m(x + g) \pm r\sqrt{1 + m^2}$
- **Tangency Condition ($y = mx + c$):** $c^2 = r^2(1 + m^2)$
- **Chord Length:** $L = 2\sqrt{r^2 - d^2}$ (d = dist to center)
- **Power of Point P:** $d^2 - r^2$ (d = dist P to center) (Useful for determining if a point is inside or outside)
- **Secant Theorem:** $PA \cdot PB = PC \cdot PD$ (AB and CD are 2 lines drawn from point P over the circle)
- **Sector Area:** $\frac{1}{2}r^2\theta$ (θ in rads)
- **Segment Area (Cut by chord):** $\frac{1}{2}r^2(\theta - \sin\theta)$

12.0.21 Points & 2D Line

- **Dist. point (x_1, y_1) to line $ax+by+c = 0$:**

$$d = \frac{|ax_1 + by_1 + c|}{\sqrt{a^2 + b^2}}$$

- **Dist. between parallel lines:** $d = \frac{|c_1-c_2|}{\sqrt{a^2+b^2}}$
- **Shoelace Area:** $\frac{1}{2}|\sum(x_iy_{i+1} - x_{i+1}y_i)|$
- **foot of Perpendicular** (h, k):

$$\frac{h - x_1}{a} = \frac{k - y_1}{b} = -\frac{ax_1 + by_1 + c}{a^2 + b^2}$$

- **Reflection (h, k):** Replace $-(...)$ with $-2(...)$ above.
- **Rotation of Axes (CCW θ):** $x = X\cos\theta - Y\sin\theta, \quad y = X\sin\theta + Y\cos\theta$

12.0.22 Vector

- **Direction Cosines:** $l = \cos\alpha, m = \cos\beta, n = \cos\gamma$. $l^2 + m^2 + n^2 = 1$.
- **Dot Product:** $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$
- **Projection of a on b:** $(\mathbf{a} \cdot \hat{b})\hat{b}$
- **Cross Product:** Area $\triangle = \frac{1}{2}|\mathbf{a} \times \mathbf{b}|$.
- **Scalar Triple (Vol):** $[\mathbf{abc}] = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$. Tet Vol $= \frac{1}{6}|[\ldots]|$.
- **Vector Triple:** $\mathbf{a}\times(\mathbf{b}\times\mathbf{c}) = (\mathbf{a}\cdot\mathbf{c})\mathbf{b} - (\mathbf{a}\cdot\mathbf{b})\mathbf{c}$

12.0.23 3D Line

Line 1: $\mathbf{r} = \mathbf{a}_1 + \lambda\mathbf{b}_1$. Line 2: $\mathbf{r} = \mathbf{a}_2 + \mu\mathbf{b}_2$.

- **Angle Between Lines:** $\cos\theta = \frac{|\mathbf{b}_1\cdot\mathbf{b}_2|}{|\mathbf{b}_1||\mathbf{b}_2|}$
- **Condition for Intersection:** Scalar Triple Product $[\mathbf{a}_2 - \mathbf{a}_1, \mathbf{b}_1, \mathbf{b}_2] = 0$.
- **Intersection Point:** Solve $\mathbf{a}_1 + \lambda\mathbf{b}_1 = \mathbf{a}_2 + \mu\mathbf{b}_2$ for λ, μ.
- **Shortest Dist (Skew):** $D = \left|\frac{(\mathbf{a}_2-\mathbf{a}_1)\cdot(\mathbf{b}_1\times\mathbf{b}_2)}{|\mathbf{b}_1\times\mathbf{b}_2|}\right|$
- **Dist Between Parallel Lines:** $D = \frac{|(\mathbf{a}_2-\mathbf{a}_1)\times\mathbf{b}|}{|\mathbf{b}|}$
- **Dist Point P to Line:** $D = \frac{|(\mathbf{p}-\mathbf{a})\times\mathbf{b}|}{|\mathbf{b}|}$
- **Foot of Perp F from P:** $\mathbf{f} = \mathbf{a} + \left(\frac{(\mathbf{p}-\mathbf{a})\cdot\mathbf{b}}{|\mathbf{b}|^2}\right)\mathbf{b}$
- **Image P' in Line:** $\mathbf{p}' = 2\mathbf{f} - \mathbf{p}$.

12.0.24 Plane

Through $\mathbf{a}$, normal $\mathbf{n}$. Eq: $(\mathbf{r} - \mathbf{a}) \cdot \mathbf{n} = 0$.

- **Cartesian:** $Ax + By + Cz + D = 0$. Normal (A, B, C).
- **Dist Point P to Plane:** $D = \frac{|Ax_1+By_1+Cz_1+D|}{\sqrt{A^2+B^2+C^2}}$
- **Angle Between Planes:** $\cos\theta = \frac{|\mathbf{n}_1\cdot\mathbf{n}_2|}{|\mathbf{n}_1||\mathbf{n}_2|}$
- **Angle Between Line & Plane:** $\sin\phi = \frac{|\mathbf{b}\cdot\mathbf{n}|}{|\mathbf{b}||\mathbf{n}|}$
- **Intersection (Line of two planes):** Direction $\mathbf{v} = \mathbf{n}_1 \times \mathbf{n}_2$.
- **Family of Planes:** $P_1 + \lambda P_2 = 0$ (Planes through intersection line).
- **Foot of Perp from P:** $\frac{x-x_1}{A} = \frac{y-y_1}{B} = \frac{z-z_1}{C} = \frac{-(Ax_1+By_1+Cz_1+D)}{A^2+B^2+C^2}$
- **Image of Point in Plane:** Use -2 instead of -1 above.
- **Plane Bisectors:** $\frac{A_1x+B_1y+C_1z+D_1}{\sqrt{A_1^2+B_1^2+C_1^2}} = \pm\frac{A_2x+B_2y+C_2z+D_2}{\sqrt{A_2^2+B_2^2+C_2^2}}$

12.0.25 Sphere

- **Standard:** $(x-u)^2+(y-v)^2+(z-w)^2 = r^2$.
- **Diameter Form:** $(x - x_1)(x - x_2) + (y - y_1)(y - y_2) + (z - z_1)(z - z_2) = 0$.
- **Tangent Plane at P:** $(\mathbf{r} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) = r^2$.
- **Circle of Intersection:** Radius $R = \sqrt{r^2 - d^2}$.
- **Spherical Cosine Rule:** $\cos a = \cos b\cos c + \sin b\sin c\cos A$.

12.0.26 Solid

- **Cone:** $V = \frac{1}{3}\pi r^2h$, $SA = \pi r(r + l)$
- **Sphere:** $V = \frac{4}{3}\pi r^3$, $SA = 4\pi r^2$
- **Spherical Cap:** $SA = 2\pi rh$. $V = \frac{1}{3}\pi h^2(3r - h)$.
- **Frustum (Cone):** $V = \frac{1}{3}\pi h(R^2 + r^2 + Rr)$. $LSA = \pi(R + r)l$.
- **Pyramid:** $V = \frac{1}{3}$(Base Area) $\times h$. $LSA = \frac{1}{2}$(Perimeter) $\times l$.
- **Regular Tetrahedron:** Side a. $H = a\sqrt{2/3}$. $V = \frac{a^3}{6\sqrt{2}}$. Face Angle $\cos\theta = 1/3$.

12.0.27 Trigonometry

- $\sin(A \pm B) = \sin A\cos B \pm \cos A\sin B$
- $\cos(A \pm B) = \cos A\cos B \mp \sin A\sin B$
- $\cos 2A = \cos^2 A - \sin^2 A = 2\cos^2 A - 1$
- $\tan^{-1} x \pm \tan^{-1} y = \tan^{-1}\left(\frac{x\pm y}{1\mp xy}\right)$ *(Note: For $+$, if $xy > 1$, add π to the result.)*
- $2\tan^{-1} x = \tan^{-1}\left(\frac{2x}{1-x^2}\right) = \sin^{-1}\left(\frac{2x}{1+x^2}\right) = \cos^{-1}\left(\frac{1-x^2}{1+x^2}\right)$
- $\tan^{-1} x + \cot^{-1} x = \frac{\pi}{2}$